Logical agents

AIMA 3rd ed. - Chapter 7

Outline

♢ Wumpus world (intro. to reasoning with an example game requiring reasoning)

♢ Knowledge base agents

♢ Representation, Reasoning, and Logic

♢ Propositional (Boolean) logic

♢ Logic in general—models and entailment

♢ Equivalence, validity, satisfiability

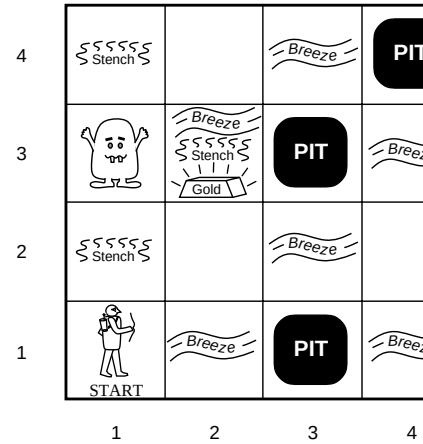
♢ Inference rules and theorem proving
  - forward chaining
  - backward chaining
  - resolution

Wumpus World PAGE description - 6.2

Percepts Breeze, Glitter, Smell

Actions Left turn, Right turn,
Forward, Grab, Release, Shoot

Goals Get gold back to start
without entering pit or wumpus square



Environment

- Squares adjacent to wumpus are smelly
- Squares adjacent to pit are breezy
- Glitter if and only if gold is in the same square
- Shooting kills the wumpus if you are facing it
- Shooting uses up the only arrow
- Grabbing picks up the gold if in the same square
- Releasing drops the gold in the same square

Wumpus world characterization - Review

Is the world deterministic??

Is the world fully accessible??

Is the world static??

Is the world discrete??

Wumpus world characterization - Review

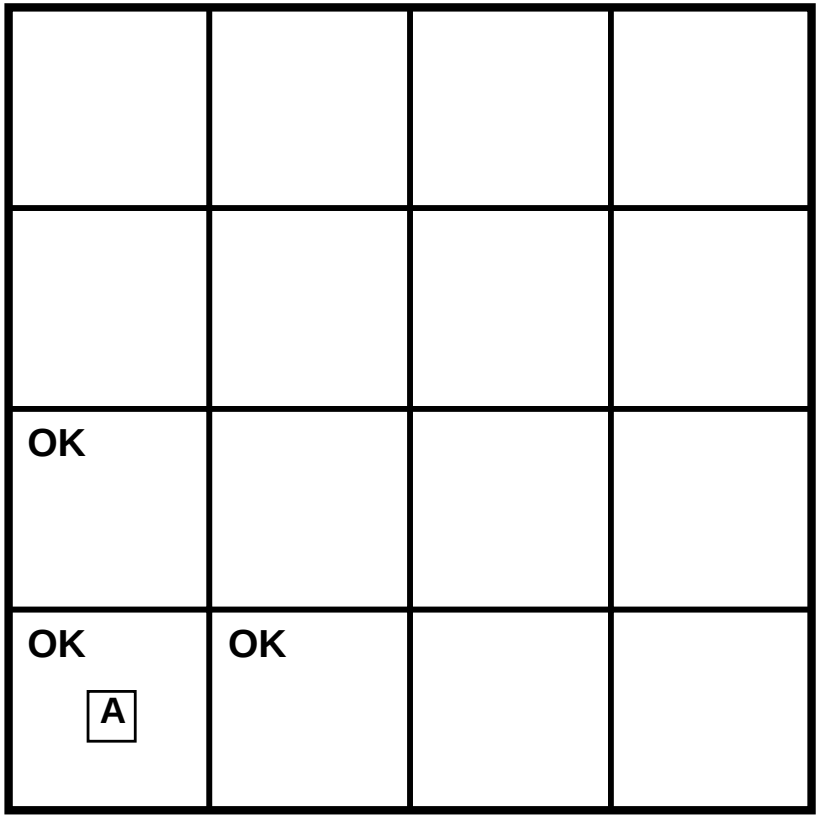
Is the world deterministic?? Yes—outcomes exactly specified

Is the world fully accessible?? No—only local perception

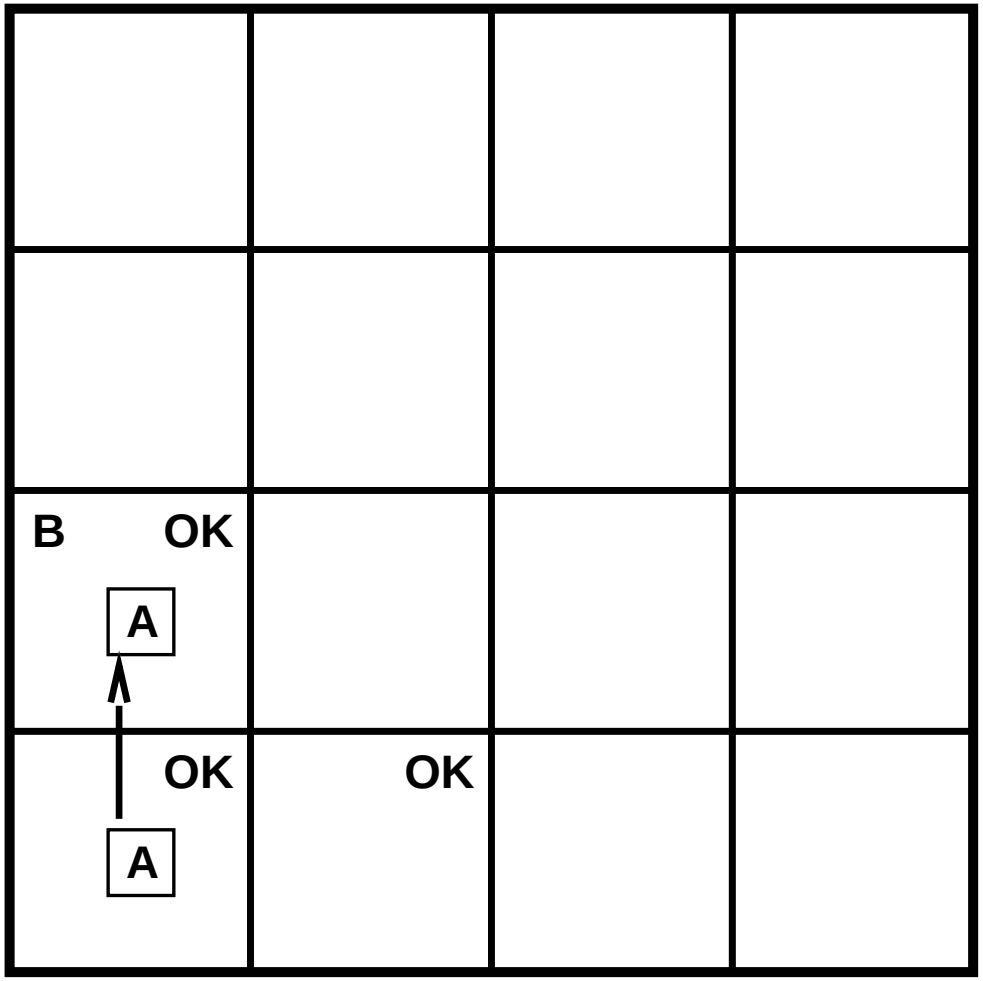
Is the world static?? Yes—Wumpus and Pits do not move

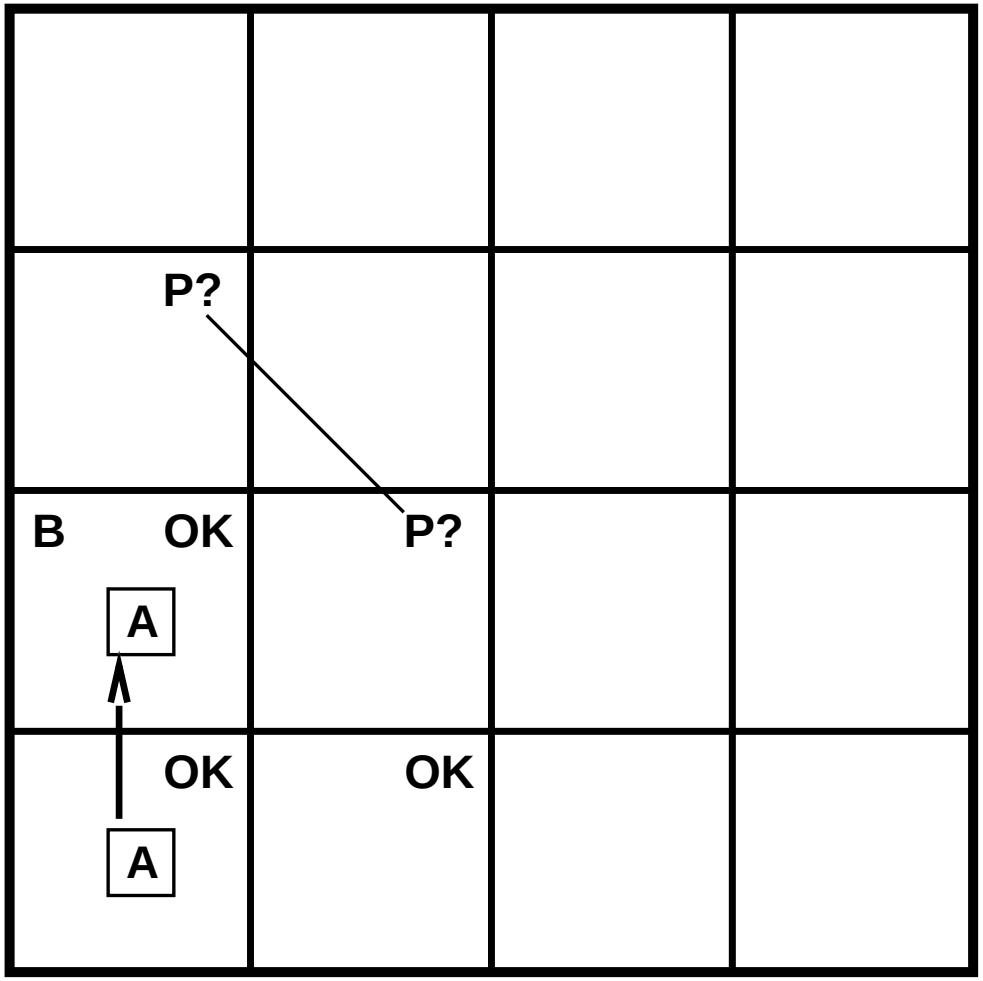
Is the world discrete?? Yes

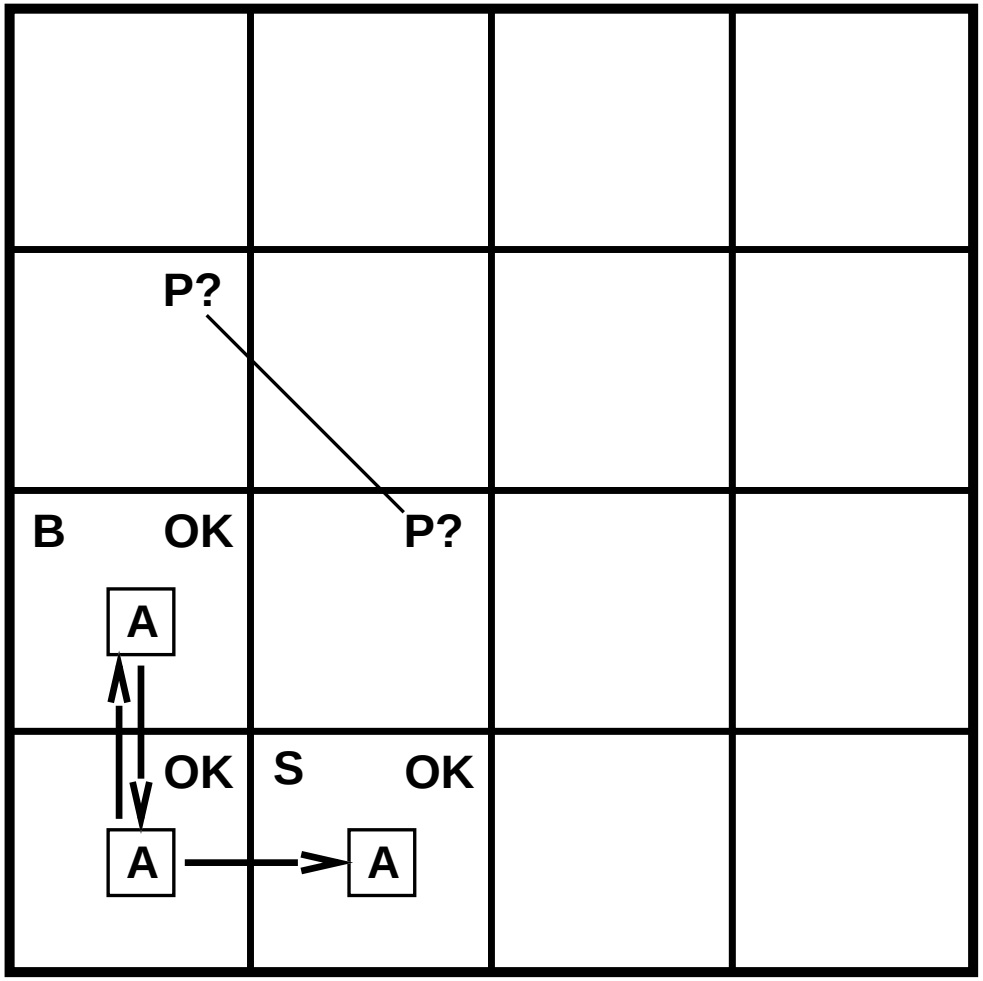
Exploring a wumpus world

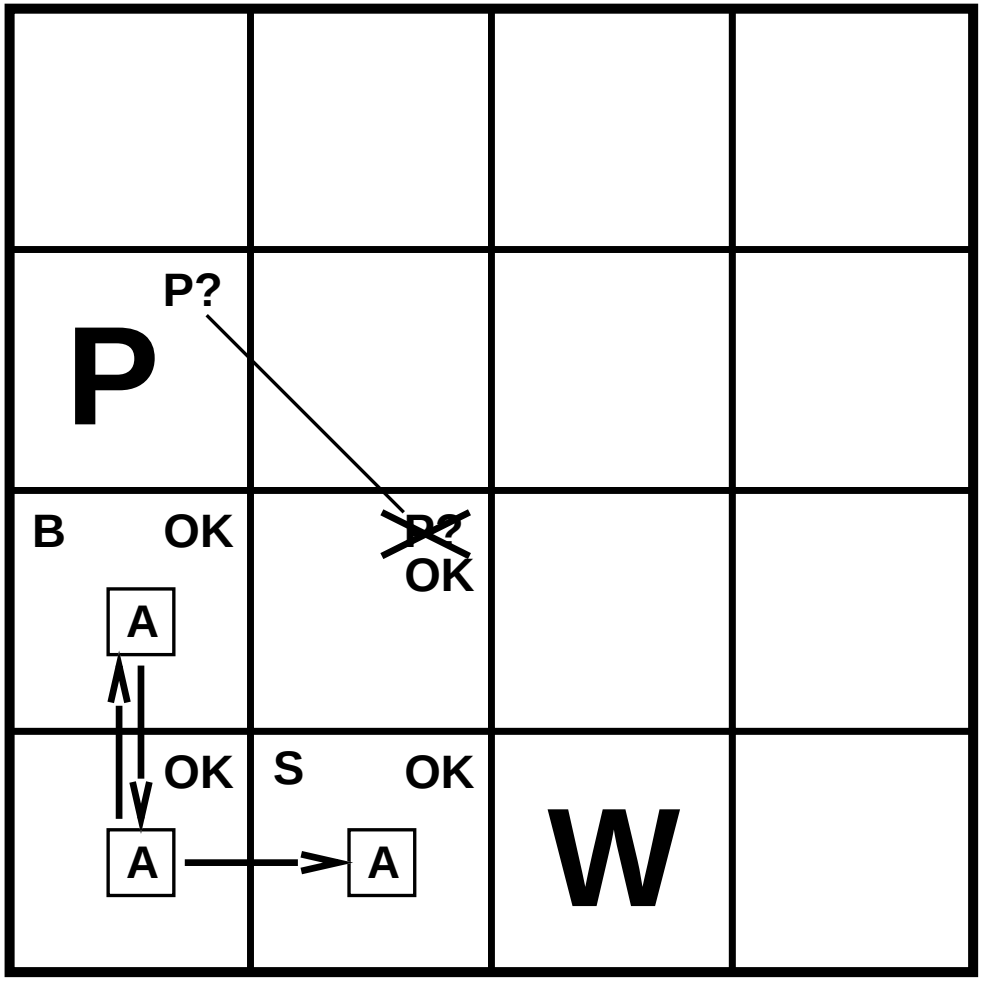


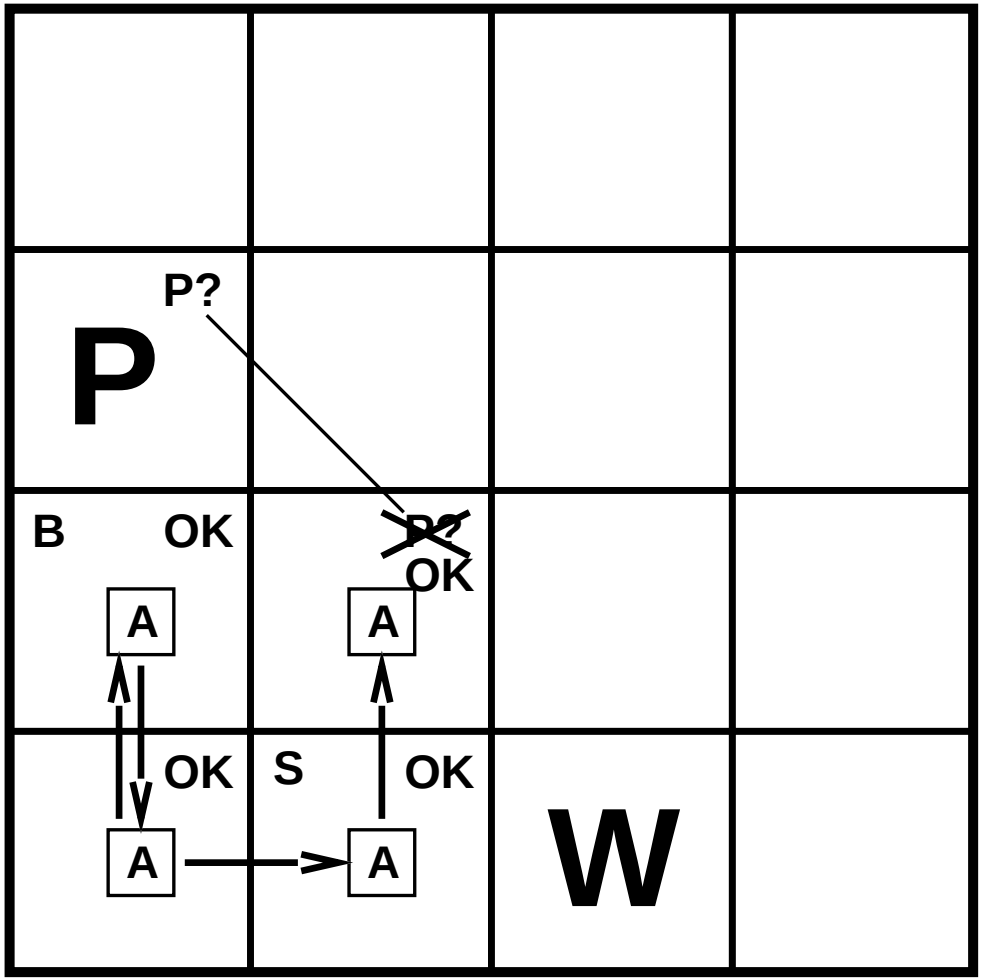
After percept [None, None, None, None, None]
[Stench Brz. Glitter Bump Scream]

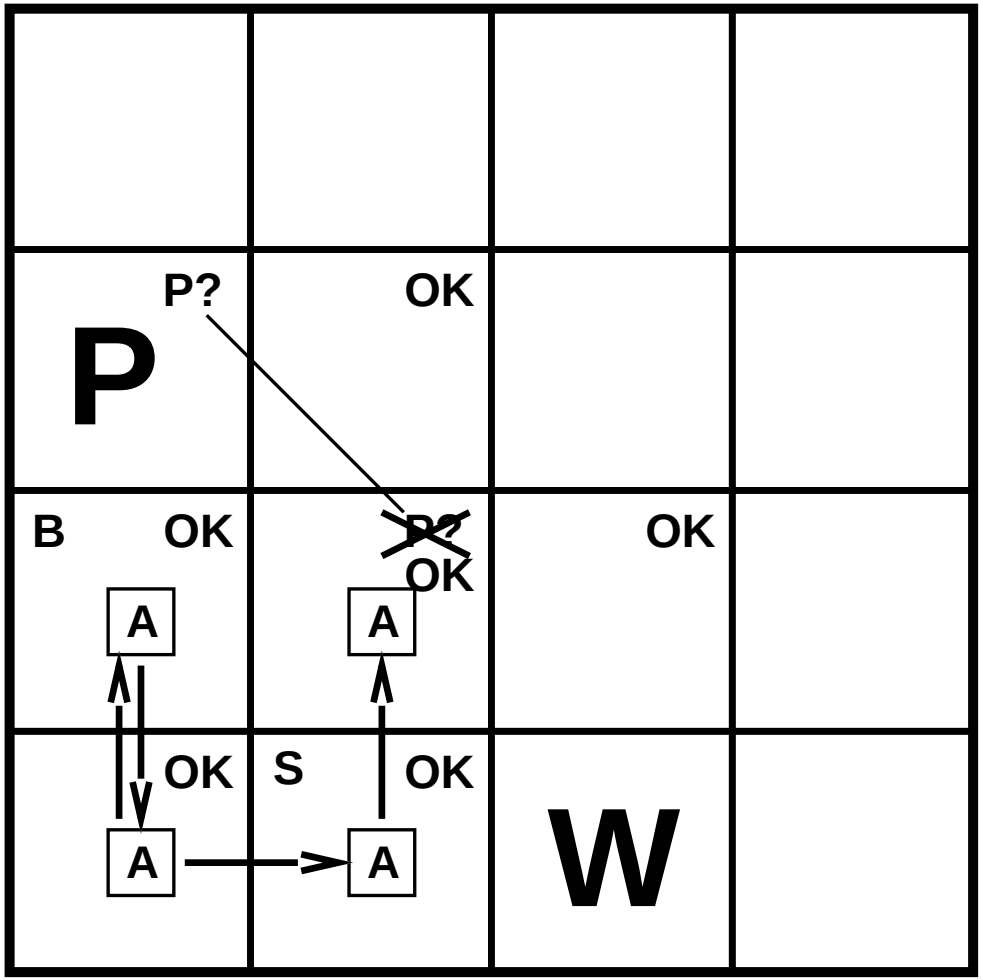

B
OK
A
OK
OK
A

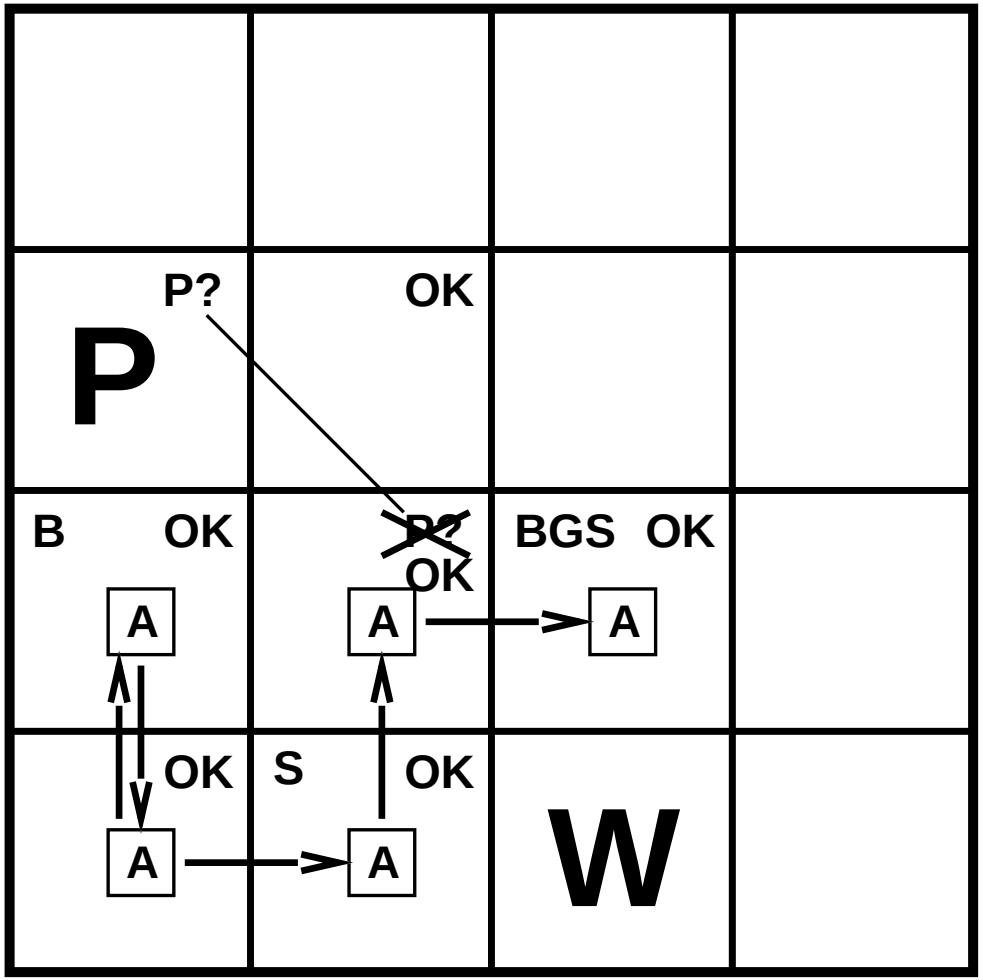

P?
B
OK
P?
A
OK
OK
A


P?
B
OK
P?
A
OK
S
OK
A
A


P?
P
B
OK
OK
A
OK
S
OK
A
A
W


P?
P
B
OK
P?
OK
A
A
OK
S
OK
A
A
W

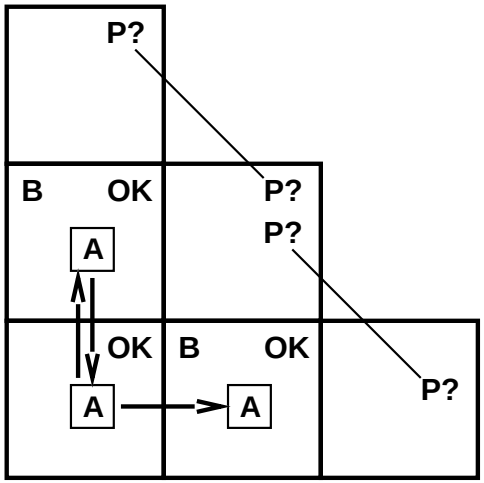

P?
OK
P
B
OK
OK
OK
A
A
OK
S
OK
A
A
W


P?
OK
P
B
OK
P?
OK
BGS
OK
A
A
A
OK
S
OK
A
A
W

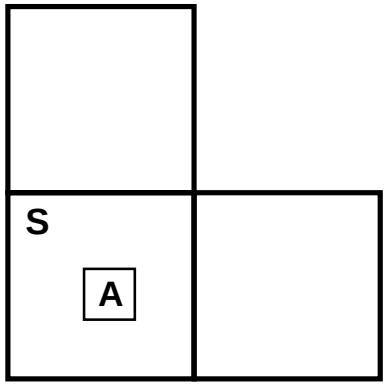
Uncertainty

There may be situations where there are no guaranteed safe moves. In that case, we will consider the **likelihoods of events** (later).

But for now, we assume that we are dealing with problems that can be solved with logical thinking and there is no chance involved (i.e. no remaining uncertainty).



Breeze in (1,2) and (2,1)
$\Rightarrow$ no safe actions



Smell in (1,1)
$\Rightarrow$ cannot move

Inference in computers

♢ How can we get an agent do what we did? That is, how can it know whether it is OK to move to [2,1]?

♢ Notice that an exhaustive search is not an option here because the agent doesnt want to die.

♢ The agent needs a formal inference mechanism to derive valid conclusions even when it does not know what the interpretation is (the computer does not need to know the meaning of OK, or Wumpus or Pit...)

Representation, Reasoning, and Logic

We will use a formal logic to extend the capacity of our agents by endowing them with the capacity for **general logical reasoning.**

Formal logic: formal languages for representing information such that conclusions can be drawn.

Knowledge based agents

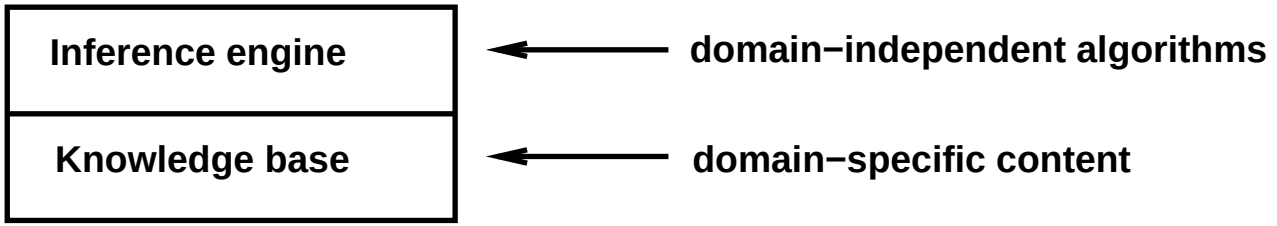
A knowledge based agent needs to know the current state of the world, how to infer unseen properties of the world, how the world evolves over time, what it wants to achieve, and what its own actions do.

Specifically, the agent must be able to:

- Represent states, actions, etc.
- Incorporate new percepts
- Deduce hidden properties of the world
- Update internal representations of the world
- Deduce appropriate actions

Knowledge bases - 6.1

Knowledge base is a set of **representations of facts (sentences)** about the world, expressed in **knowledge representation language.****



Declarative approach to building an agent (or other system):

TELL it what it needs to know

Then it can ASK itself what to do—answers should follow from the KB

Knowledge bases

Agents can be viewed at the knowledge level

i.e., what they know, **regardless of how implemented**

The implementation level may represent the knowledge in various forms:

e.g. UcakSeferi(Istanbul,Samsun) can be represented as a string in a list of strings or as 1 in a 2D table indexed by location pairs etc.

But we don't care about the implementation details of the KB.

Entailment

We are interested in new sentences that we can infer from the knowledgebase and the new percepts. E.g. seeing that there is no breeze in [1,1], can we infer that there is no wumpus in [1,2]? We will use the word entailment, which means that one thing **follows from** another.

Knowledge base KB entails sentence α ($KB \models \alpha$) if and only if α is true in all worlds where KB is true

E.g., the KB containing "Ali and Ayse are good students"
entails "Ayse is a good student"

E.g., $x + y = 4$ entails $4 = x + y$

Entailment is a relationship between sentences (i.e., **syntax**)
that is based on **semantics**

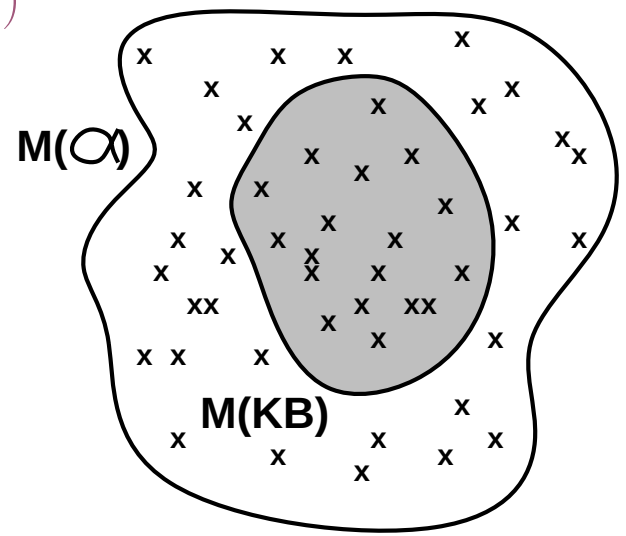
Models

We say that m **is a model of** a sentence α if α is true in m

$M(\alpha)$ is the set of all models of α

Then $KB \models \alpha$ if and only if $M(KB) \subseteq M(\alpha)$

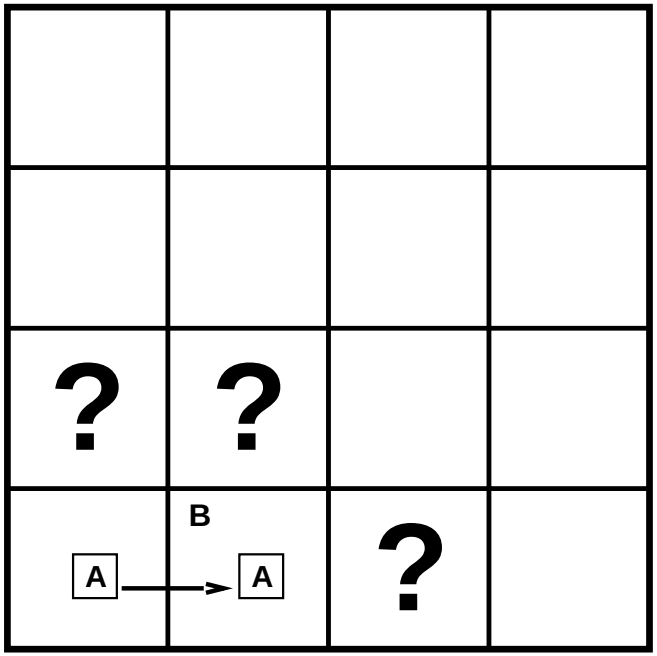
E.g. $KB =$ Ali and Ayse are good students
$\alpha =$ Ayse is a good student



Difference between $world$ and $model$:
"model" is used interchangeably with "world" in the general sense;
but a "model" of a particular sentence is a "world" in which that sentence is True)

Entailment in the wumpus world

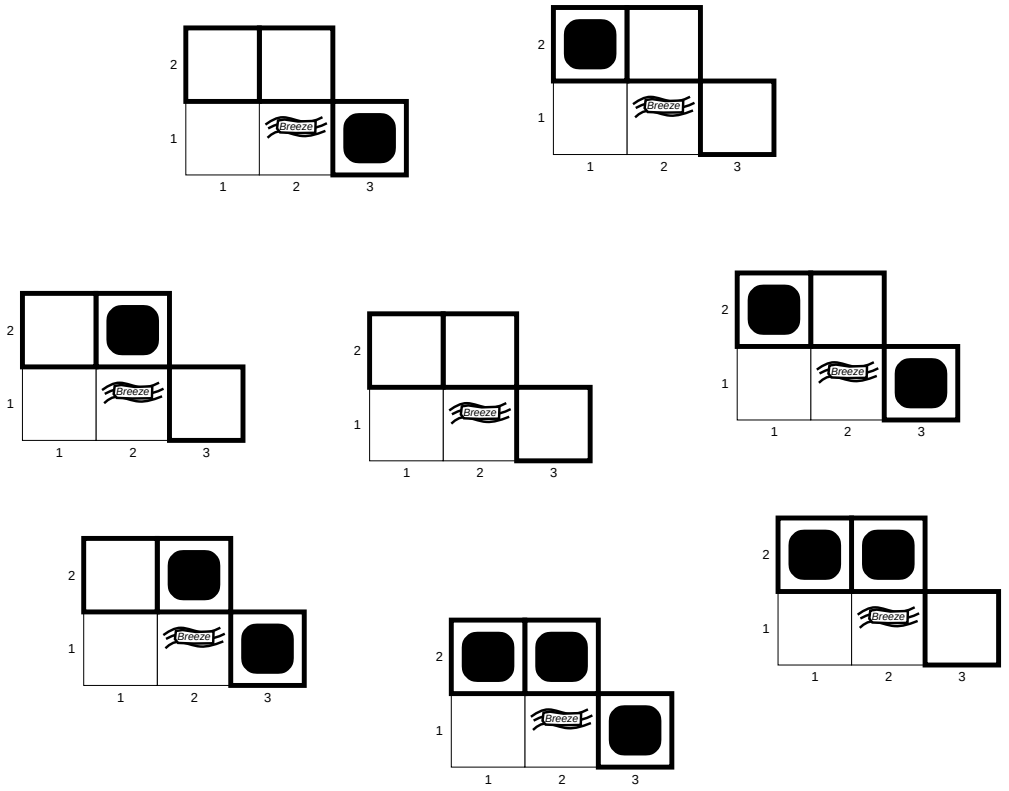
Situation after detecting nothing in [1,1], moving right, breeze in [2,1]



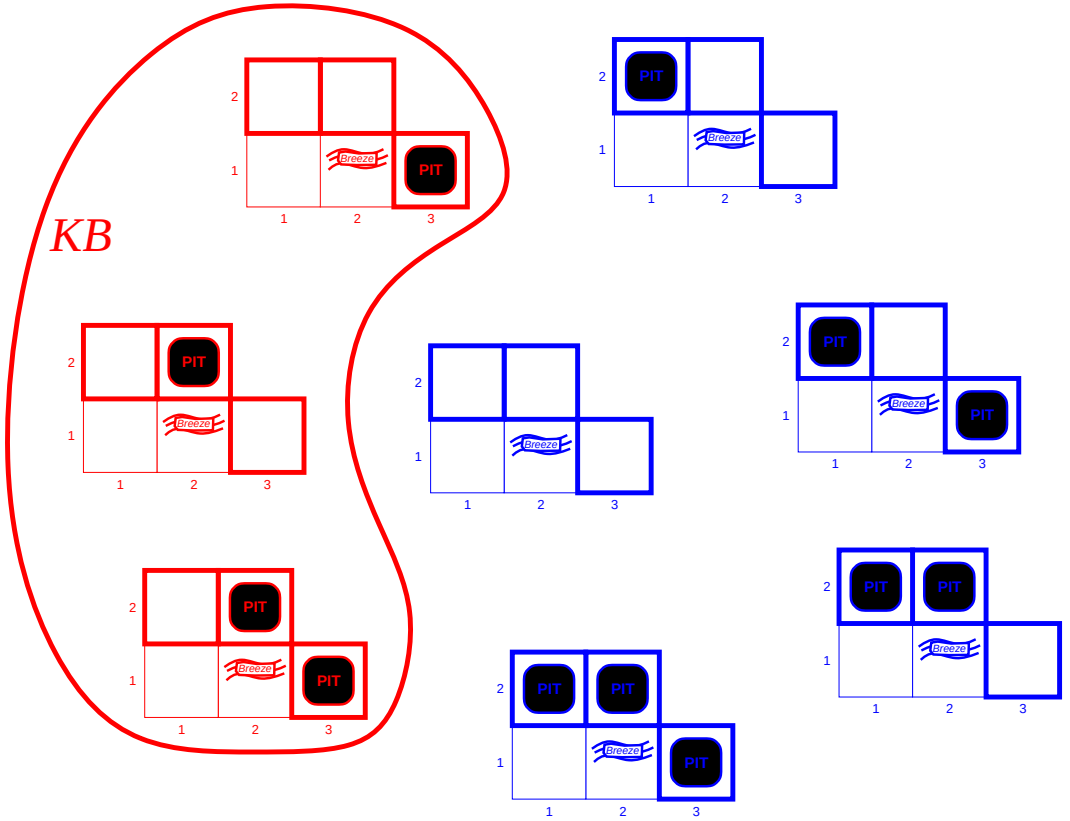
Consider all possible assignments to ?s assuming only pits

3 Boolean choices $\Rightarrow$ 8 possible worlds

Wumpus models

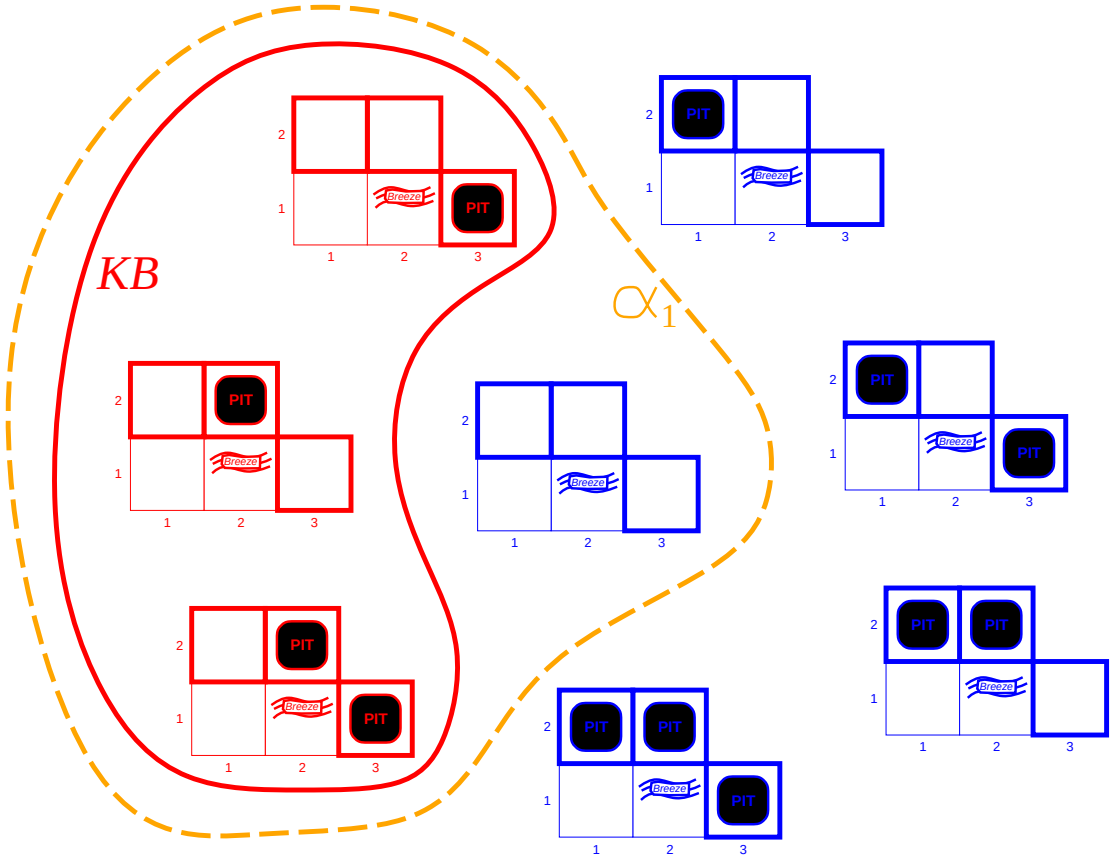


Wumpus models



KB = wumpus-world rules + observations

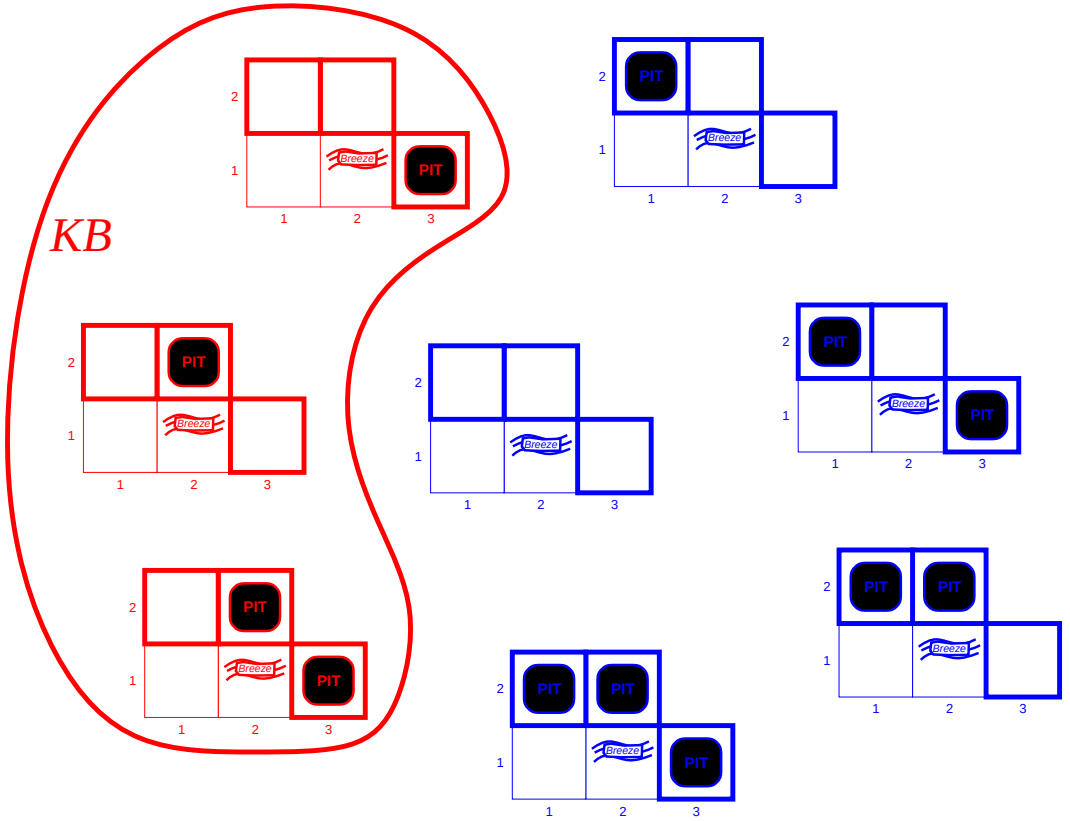
Wumpus models



KB = wumpus-world rules + observations

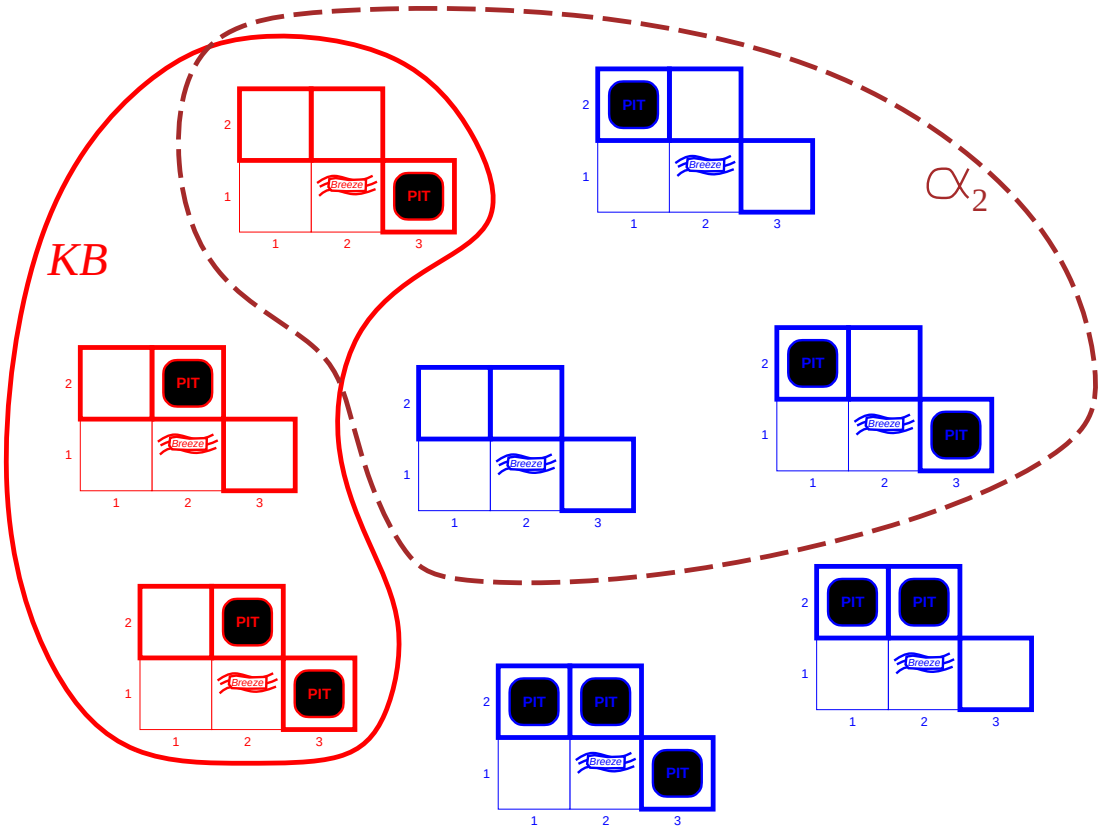
α_1 = "[1,2] is safe", $KB \models \alpha_1$, proved by model checking

Wumpus models



KB = wumpus-world rules + observations

Wumpus models



KB = wumpus-world rules + observations

α_2 = "[2,2] is safe", $KB \not\models \alpha_2$

Representation

We know of some knowledge representation languages:

♢ Programming languages

In a programming language you can say that "there is a pit in [2,2]" by world[2,2] = pit, but you cannot say "either there is a pit in [2,1] *or* in [3,2]" or that "there is a wumpus in *some* square"

♢ Propositional logic

♢ First-order logic

♢ Natural languages are very **expressive**, but also too flexible/ambiguous

We will concentrate on first-order logic: basis of most representation schemes in AI. It is expressive enough to say almost anything of interest, and for which there exists a sound and complete inference procedure (complete: the procedure will answer any question whose answer follows from what is known by the KB.)

Logic

Logic is a formal language for representing information
such that conclusions can be drawn

Syntax defines the elements and possible sentences in the language

Semantics define the "meaning" of sentences;

Consider the language of arithmetic:

Syntax:

$x + 2 \geq y$ is a sentence; $x + y +$ is not a sentence

Semantics:

$x + 2 \geq y$ is true iff the number $x + 2$ is no less than the number y

$x + 2 \geq y$ is true in a world where $x = 7,\ y = 1$
$x + 2 \geq y$ is false in a world where $x = 0,\ y = 6$

Propositional logic: Syntax

Propositional logic is the simplest logic:

A sentence can be one of the following:

♢ Constant propositions True or False

♢ Propositions such as P or Q which may have value True or False

♢ Sentences formed by simpler ones using logical connectives:

If S is a sentence, $\neg S$ is a sentence

If S_1 and S_2 is a sentence, $S_1 \wedge S_2$ is a sentence

If S_1 and S_2 is a sentence, $S_1 \vee S_2$ is a sentence

If S_1 and S_2 is a sentence, $S_1 \Rightarrow S_2$ is a sentence

If S_1 and S_2 is a sentence, $S_1 \Leftrightarrow S_2$ is a sentence

Propositional logic: Semantics

Assume you are given a model, m, specifying the truth settings (true/false) for each proposition symbol:

E.g.

A	B	C
$True$	$True$	$False$

Rules for **evaluating truth with respect to a model** m:

$\neg S$	is true iff		S	is false				
$S_1 \wedge S_2$	is true iff		S_1	is true and		S_2	is true	
$S_1 \vee S_2$	is true iff		S_1	is true or		S_2	is true	
$S_1 \Rightarrow S_2$	is true iff		S_1	is false or		S_2	is true	
i.e.,	is false iff		S_1	is true and		S_2	is false	
$S_1 \Leftrightarrow S_2$	is true iff	$S_1 \Rightarrow S_2$		is true and	$S_2 \Rightarrow S_1$			is true

Propositional logic: Semantics

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
False	*False*	*True*	*False*	*False*	*True*	*True*
False	*True*	*True*	*False*	*True*	*True*	*False*
True	*False*	*False*	*False*	*True*	*False*	*False*
True	*True*	*False*	*True*	*True*	*True*	*True*

(P implies Q) is the only one a bit unintuitive: if P is false, the implication is basically irrelevant. So, simply look to see if Q is true whenever P is.

Logical equivalence

Two sentences are logically equivalent iff true in same models:
$\alpha \equiv \beta$ if and only if $\alpha \models \beta$ and $\beta \models \alpha$

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Validity and Satisfiability

A sentence is valid (is a tautology) if it is true in all models

e.g., $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

A sentence is satisfiable if it is true in some model

e.g., $A \vee B$, C

A sentence is unsatisfiable if it is true in no models

e.g., $A \wedge \neg A$

Validity is connected to inference:

$KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid

Satisfiability is connected to inference:

$KB \models \alpha$ if and only if $(KB \wedge \neg\alpha)$ is unsatisfiable

i.e., prove α by *reductio ad absurdum* - we will see this later

Validity and Inference

Truth tables can be used to test for valid sentences.

E.g. Show that $((P \vee H) \wedge \neg H) \Rightarrow P$ is a valid sentence. How?

Validity and Inference

Show that $((P \vee H) \wedge \neg H) \Rightarrow P$ is a valid sentence.

P	H	$P \vee H$	$(P \vee H) \wedge \neg H$	$((P \vee H) \wedge \neg H) \Rightarrow P$
False	*False*	*False*	*False*	*True*
False	*True*	*True*	*False*	*True*
True	*False*	*True*	*True*	*True*
True	*True*	*True*	*False*	*True*

If the sentence is True in all possible worlds (i.e., rows indicating the truth assignments to all the variables), it is Valid.

Proof/Inference methods

Assume that we first knew that "there is a Pit in 2x2 or 2x1" (from the previous slide: $P \vee H)$, and then we learned "there is no Pit in 2x1" ($\neg H$).

Then we could conclude that "there is a Pit in 2x2" (P).

Proof = a sequence of inference rule applications that shows a conclusion (e.g. pit in 2x2).

Proof/Inference methods

There are roughly two kinds of proof methods:

Model checking:

- Truth table enumeration (sound and complete for propositional logic)
- Heuristic search in model space (sound but incomplete)

Application of inference rules

- Legitimate (sound) generation of new sentences from old
- Can use inference rules as operators in a standard search alg.

Propositional inference: Enumeration method

Let $\alpha = A \vee B$ and $KB = (A \vee C) \wedge (B \vee \neg C)$

Is it the case that $KB \models \alpha$?
Check all possible models—α must be true wherever KB is true

A	B	C	$A \vee C$	$B \vee \neg C$	KB	α
$False$	$False$	$False$				
$False$	$False$	$True$				
$False$	$True$	$False$				
$False$	$True$	$True$				
$True$	$False$	$False$				
$True$	$False$	$True$				
$True$	$True$	$False$				
$True$	$True$	$True$				

Propositional inference: Enumeration method

Let $\alpha = A \vee B$ and $KB = (A \vee C) \wedge (B \vee \neg C)$

Is it the case that $KB \models \alpha$?
Check all possible models: α **must be true wherever** KB **is true**

A	B	C	$A \vee C$	$B \vee \neg C$	KB	α
$False$	$False$	$False$	$False$	$True$	$False$	$False$
$False$	$False$	$True$	$True$	$False$	$False$	$False$
$False$	$True$	$False$	$False$	$True$	$False$	$True$
$False$	$True$	$True$	$True$	$True$	$True$	$True$
$True$	$False$	$False$	$True$	$True$	$True$	$True$
$True$	$False$	$True$	$True$	$False$	$False$	$True$
$True$	$True$	$False$	$True$	$True$	$True$	$True$
$True$	$True$	$True$	$True$	$True$	$True$	$True$

Note that this is equivalent to proving that the sentence
$((A \vee C) \wedge (B \vee \neg C) \Rightarrow A \vee B$ is a valid sentence!

Complexity of inference

Truth table method: 2^N rows of table for any proof involving N symbols

Linear space requirements though using Depth-First search.

PL Inference: Enumeration method

function TT-ENTAILS?(KB, α) **returns** *true* or *false*
  inputs: KB, the knowledge base, a sentence in propositional logic
    α, the query, a sentence in propositional logic

  $symbols \leftarrow$ a list of the proposition symbols in KB and α
  return TT-CHECK-ALL($KB, \alpha, symbols, [\,]$)

---

function TT-CHECK-ALL($KB, \alpha, symbols, model$) **returns** *true* or *false*
  if EMPTY?($symbols$) **then**
    if PL-TRUE?($KB, model$) **then return** PL-TRUE?($\alpha, model$)
    else return *true*
  else do
    $P \leftarrow$ FIRST($symbols$); $rest \leftarrow$ REST($symbols$)
    return TT-CHECK-ALL($KB, \alpha, rest$, EXTEND(P, *true*, $model$)) **and**
      TT-CHECK-ALL($KB, \alpha, rest$, EXTEND(P, *false*, $model$))

Figure 7.10 A truth-table enumeration algorithm for deciding propositional entailment. TT stands for truth table. PL-TRUE? returns true if a sentence holds within a model. The variable *model* represents a partial model—an assignment to only some of the variables. The function call EXTEND(*P, true, model*) returns a new partial model in which P has the value *true*.

Inference

An inference procedure that generates only entailed sentences is called **sound**

An inference procedure that generates all sentences that can be entailed is called **complete**

Formally:

$KB \vdash_i \alpha =$ sentence α can be derived from KB by procedure i

Soundness: i is sound if
whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$

Completeness: i is complete if
whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$

Rules of Inference

Instead of enumeration method, we can also use inference rules that we know are sound. We list these rules as:

$$\frac{\alpha}{\beta}$$

Notation: Whenever a sentence in the KB matches the pattern above the line (α), the inference rule can conclude the sentence below the line (β).

Complete Set of Inference rules for propositional logic

Modus Ponens (or implication elimination)

$$\frac{\alpha_1, \ldots, \alpha_n, \qquad \alpha_1 \wedge \cdots \wedge \alpha_n \Rightarrow \beta}{\beta}$$

And-elimination (from a conjunction, you can infer any of the conjuncts)

$$\frac{\alpha_1 \wedge \cdots \wedge \alpha_n}{\alpha_i}$$

Unit resolution (from a disjunction, if one of the disjuncts are false, you can infer the other one is true)

$$\frac{\alpha \vee \beta \qquad \neg\beta}{\alpha}$$

Resolution (for CNF): **complete** for propositional logic

$$\frac{\alpha \vee \beta, \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Application: Propositional inference in the Wumpus world - 6.5

Assume the following situation:

1,4	2,4	3,4	4,4
1,3 **W!**	2,3	3,3	4,3
1,2 **A** **S** **OK**	2,2 **OK**	3,2	4,2
1,1 **V** **OK**	2,1 **B** **V** **OK**	3,1 **P!**	4,1

A = *Agent*
B = *Breeze*
G = *Glitter, Gold*
OK = *Safe square*
P = *Pit*
S = *Stench*
V = *Visited*
W = *Wumpus*

How does the agent know where to go?

Entailment Using Inference Rules

$R_1 : \quad \neg P_{1,1} .$

$R_2 : \quad B_{1,1} \quad \Leftrightarrow \quad (P_{1,2} \vee P_{2,1}) .$

$R_3 : \quad B_{2,1} \quad \Leftrightarrow \quad (P_{1,1} \vee P_{2,2} \vee P_{3,1}) .$

$R_4 : \quad \neg B_{1,1} .$

$R_5 : \quad B_{2,1} .$

Let us see how these inference rules and equivalences can be used in the wumpus world. We start with the knowledge base containing R_1 through R_5, and show how to prove $\neg P_{1,2}$, that is, there is no pit in [1,2]. First, we apply biconditional elimination to R_2 to obtain

$$R_6 : \quad (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) .$$

Then we apply And-Elimination to R_6 to obtain

$$R_7 : \quad ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) .$$

Logical equivalence for contrapositives gives

$$R_8 : \quad (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})) .$$

Now we can apply Modus Ponens with R_8 and the percept R_4 (i.e., $\neg B_{1,1}$), to obtain

$$R_9 : \quad \neg(P_{1,2} \vee P_{2,1}) .$$

Finally, we apply de Morgan's rule, giving the conclusion

$$R_{10} : \quad \neg P_{1,2} \wedge \neg P_{2,1} .$$

That is, neither [1,2] nor [2,1] contains a pit.

Remember: Validity and Satisfiability

A sentence is valid (is a tautology) if it is true in all models

e.g., $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

A sentence is satisfiable if it is true in some model

e.g., $A \vee B$, C

A sentence is unsatisfiable if it is true in no models

e.g., $A \wedge \neg A$

Validity is connected to inference:

$KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid

Satisfiability is connected to inference:

$KB \models \alpha$ if and only if $(KB \wedge \neg\alpha)$ is unsatisfiable

i.e., prove α by *reductio ad absurdum* - we will see this later

Normal forms

Other approaches to inference use syntactic operations on sentences which are often expressed in standardized forms:

Horn Form (**restricted**)

conjunction of *Horn clauses* (clauses with ≤ 1 positive literal)

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

Often written as set of implications:

$B \Rightarrow A$ and $(C \wedge D) \Rightarrow B$

Disjunctive Normal Form (DNF—universal)

disjunction of $\underbrace{\textit{conjunctions}\text{ of }\textit{literals}}_{\textit{terms}}$

E.g., $(A \wedge B) \vee (A \wedge \neg C) \vee (A \wedge \neg D) \vee (\neg B \wedge \neg C) \vee (\neg B \wedge \neg D)$

Conjunctive Normal Form (CNF—universal)

conjunction of $\underbrace{\textit{disjunctions}\text{ of }\textit{literals}}_{\textit{clauses}}$

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

Resolution requires the Conjunctive Normal Form.

Every sentence can be written in CNF.

Resolution

Conjunctive Normal Form (CNF—universal)

conjunction of $\underbrace{\textit{disjunctions of literals}}_{\textit{clauses}}$

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

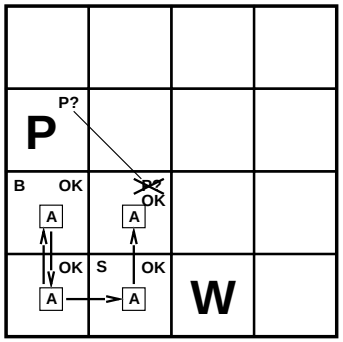
Resolution inference rule (for CNF): complete for propositional logic

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

where ℓ_i and m_j are complementary literals. E.g.,

$$\frac{P_{1,3} \vee P_{2,2}, \qquad \neg P_{2,2}}{P_{1,3}}$$

Resolution is sound and complete for propositional logic



Conversion to CNF

$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ($\vee$ over $\wedge$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Resolution algorithm

Proof by contradiction, i.e., show $KB \wedge \neg\alpha$ unsatisfiable

function PL-RESOLUTION(KB, α) **returns** *true* or *false*

$clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$
$new \leftarrow \{\,\}$
loop do
  for each C_i, C_j **in** *clauses* **do**
    $resolvents \leftarrow$ PL-RESOLVE(C_i, C_j)
    if *resolvents* contains the empty clause **then return** *true*
    $new \leftarrow new \cup resolvents$
  if $new \subseteq clauses$ **then return** *false*
  $clauses \leftarrow clauses \cup new$

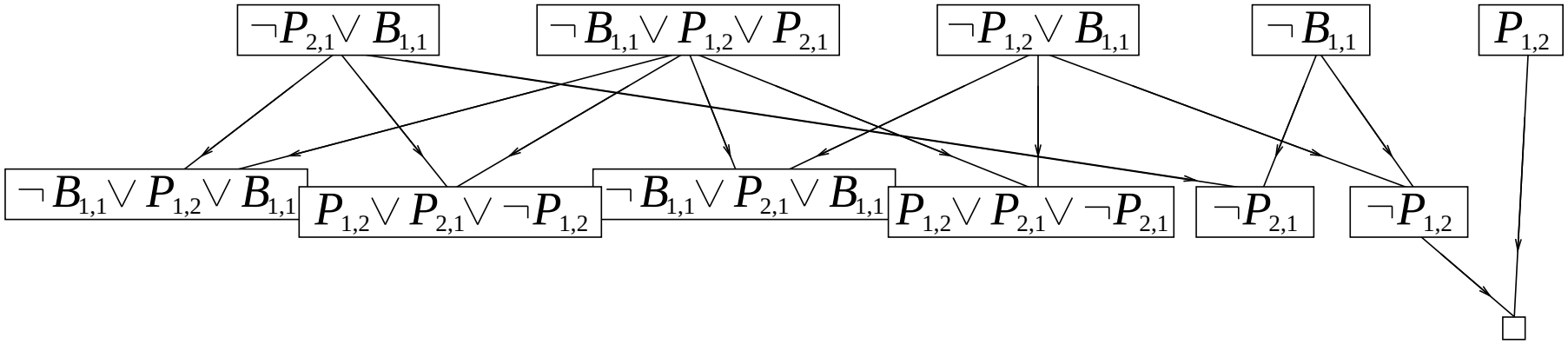
Resolution example

$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$ $\alpha = \neg P_{1,2}$

Add the negation of the sentence you want to prove (e.g. $\neg\neg P_{1,2}$) and see if you reach a contradiction:

Resolution example

$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$ $\alpha = \neg P_{1,2}$



Restricted Forms: Horn Form

The completeness of resolution makes it a very important inference method. In many practical situations however, the full power of resolution is not needed.

Horn Form (restricted): A KB is said to be in Horn Form if it is a *conjunction* of *Horn clauses*.

Horn clause =

- ♢ proposition symbol; or
- ♢ (conjunction of symbols) $\Rightarrow$ symbol

Alternative definition: a Horn clause is a disjunction of literals of which *at most one* is positive

Example KB: $C \wedge (B \Rightarrow A) \wedge (C \wedge D \wedge E \Rightarrow B)$

Definition: Body (premise) and Head (conclusion).

Restricted Forms: Horn Form

The Horn form seems very restricted, but many real databases can be written in Horn form. How?

♢ Definite clauses: with exactly one positive literal

e.g.: $(C \wedge D \Rightarrow B)$

♢ Facts: a definite clause with no negative literals simply asserts a given proposition

e.g.: C

♢ Integrity constraints: a Horn clause with no positive literals

e.g.: $\neg(W11 \wedge W22)$

Restricted Forms: Horn Form

How can you convert each sentence into an implication?

$(W11 \wedge W22) \Rightarrow False$

Restricted Forms: Horn Form

Modus Ponens is complete for Horn KBs (not in general)

$$\frac{\alpha_1, \ldots, \alpha_n, \qquad \alpha_1 \wedge \cdots \wedge \alpha_n \Rightarrow \beta}{\beta}$$

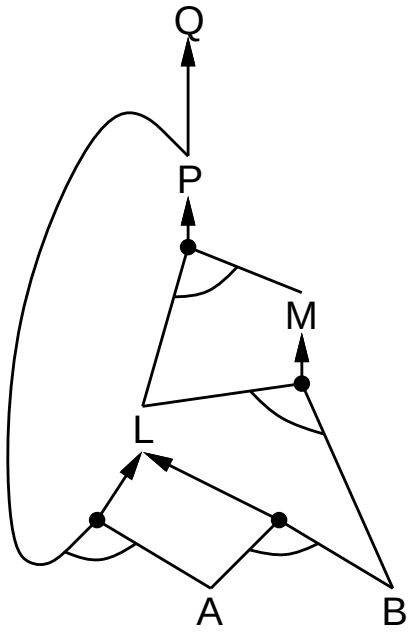
Can be used with forward chaining or backward chaining.
These algorithms are very natural and run in *linear* time in the size of the KB.

They are also sound (easy to show) and complete (slightly more complicated argument).

Forward chaining

Idea: fire any rule whose premises are satisfied in the KB,
add its conclusion to the KB, until query is found

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$

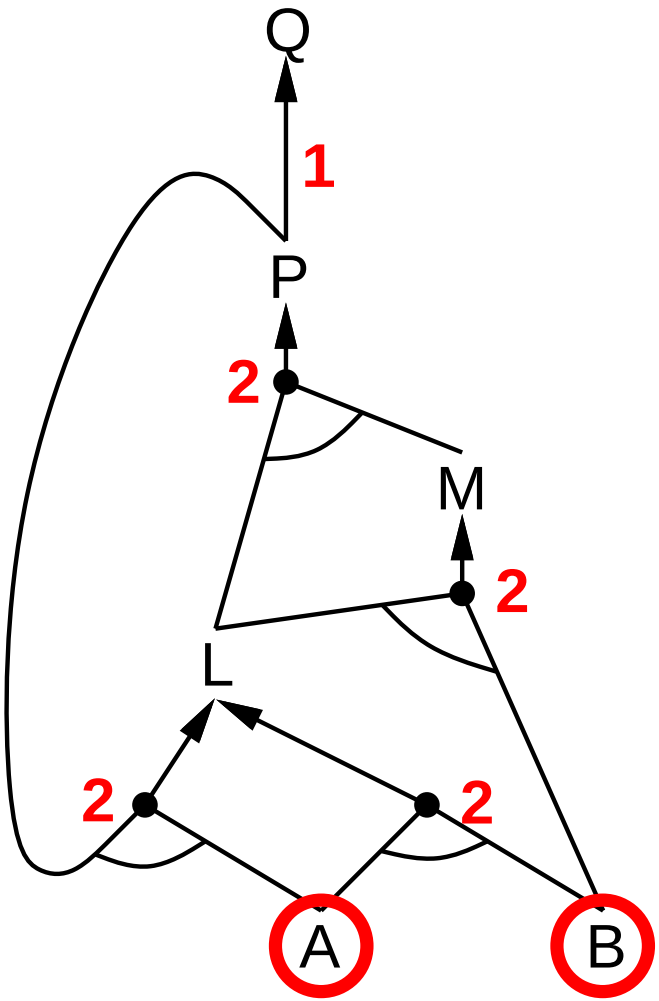


Forward chaining algorithm

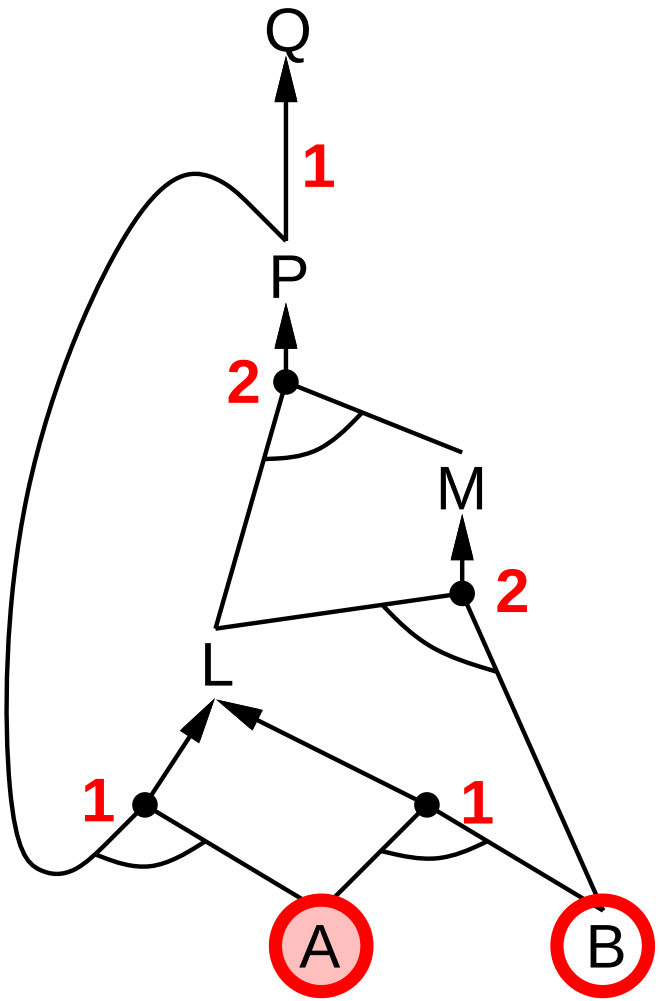
```
function PL-FC-ENTAILS?(KB, q) returns true or false
  local variables: count, a table, indexed by clause, initially the number of premises
                   inferred, a table, indexed by symbol, each entry initially false
                   agenda, a list of symbols, initially the symbols known to be true

  while agenda is not empty do
      p ← POP(agenda)
      unless inferred[p] do
          inferred[p] ← true
          for each Horn clause c in whose premise p appears do
              decrement count[c]
              if count[c] = 0 then do
                  if HEAD[c] = q then return true
                  PUSH(HEAD[c], agenda)
  return false
```

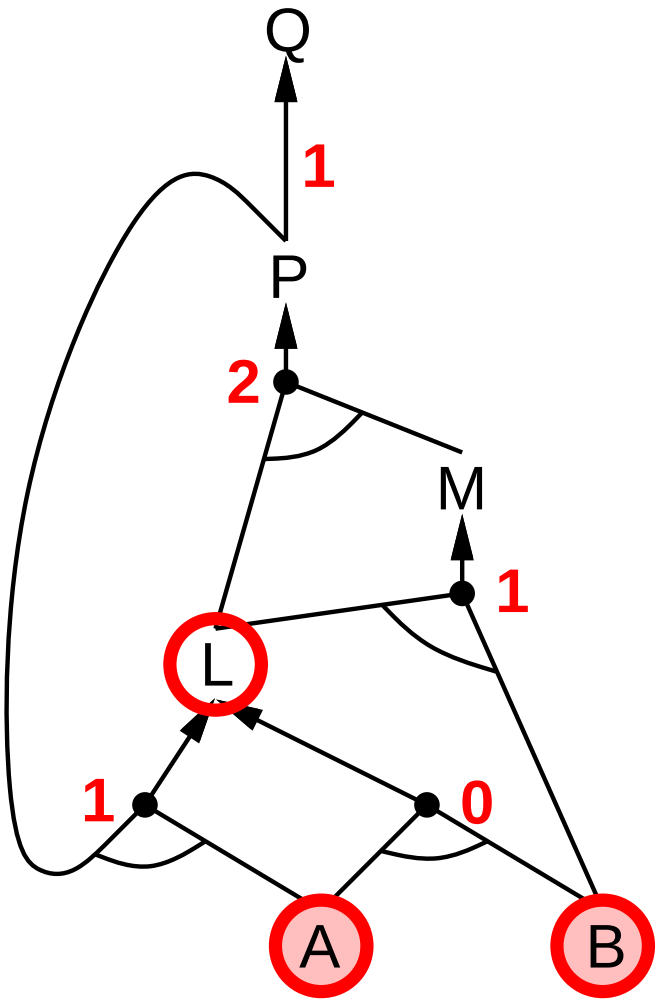
Forward chaining example



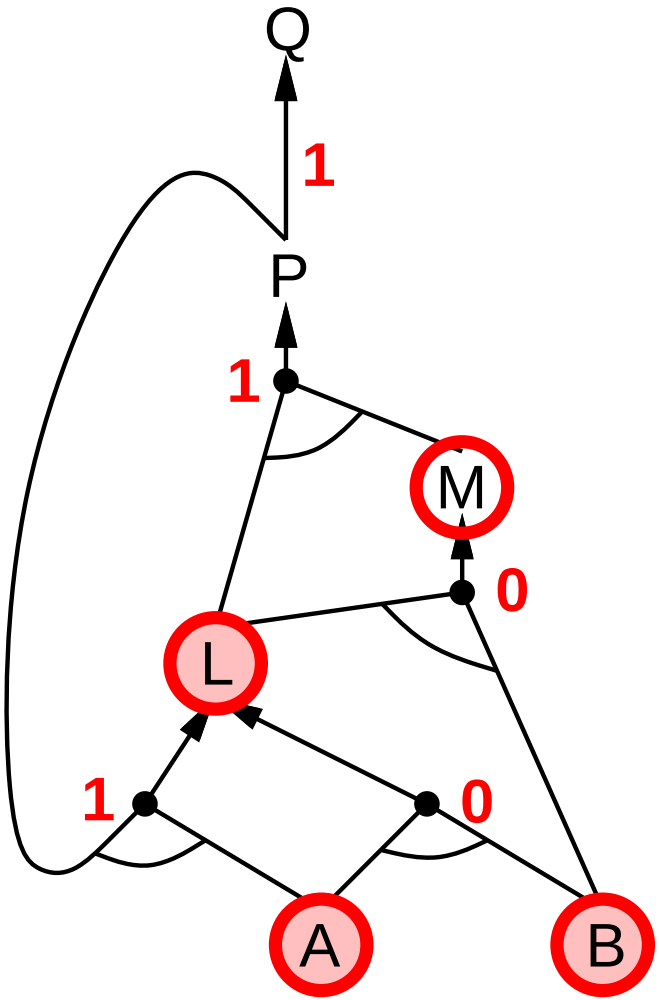
Forward chaining example



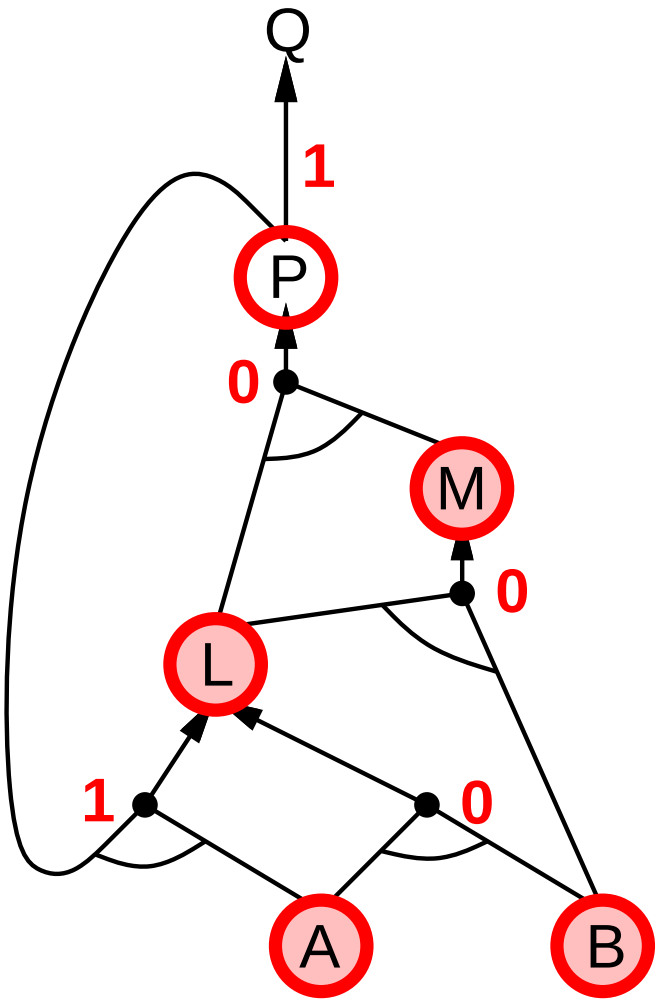
Forward chaining example



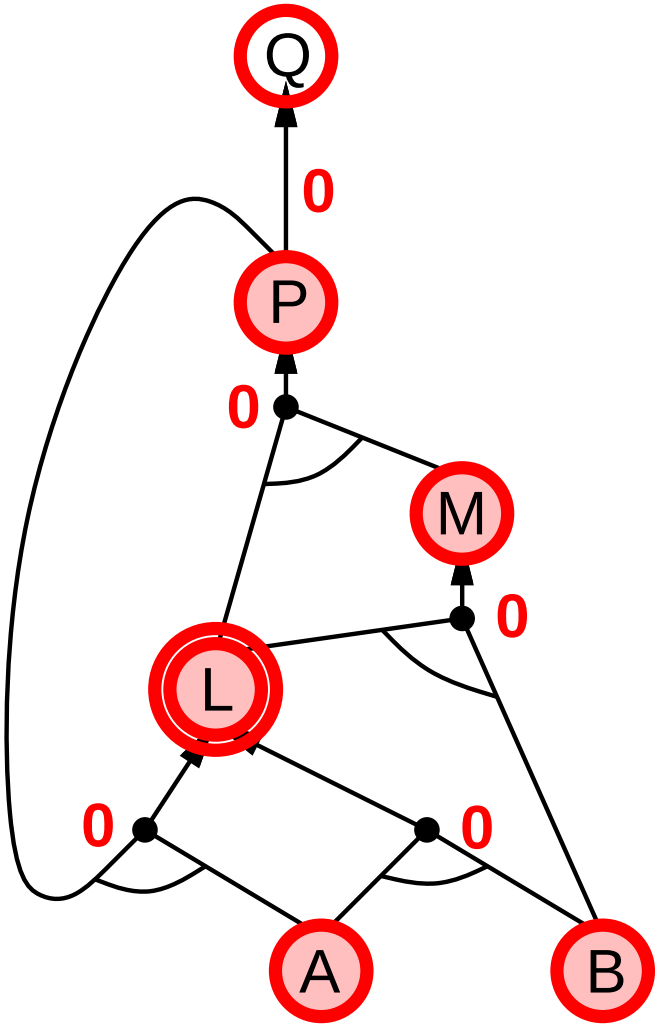
Forward chaining example



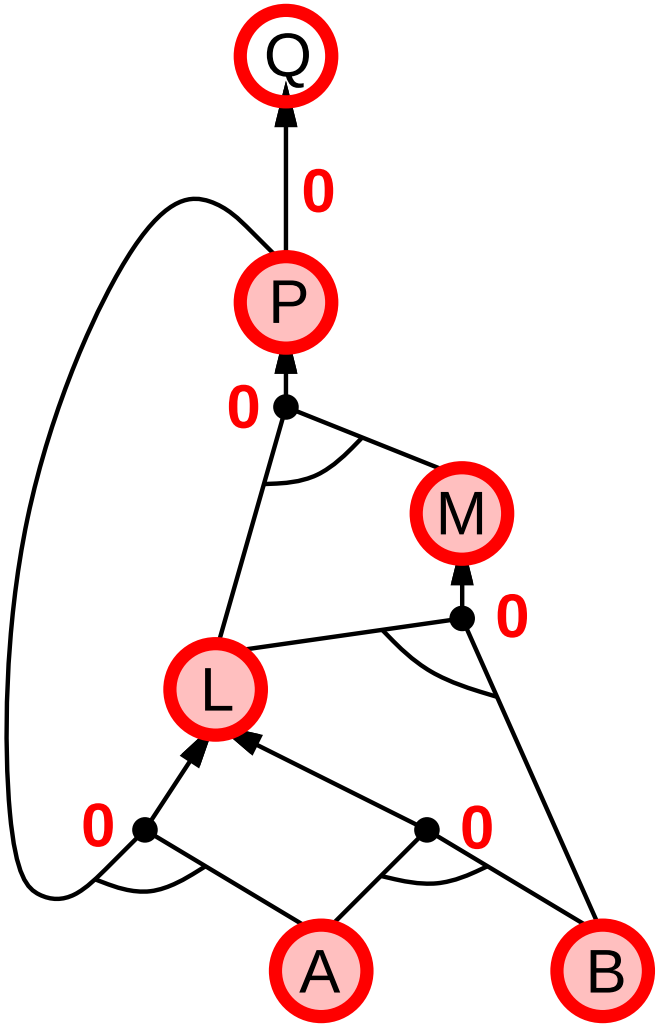
Forward chaining example



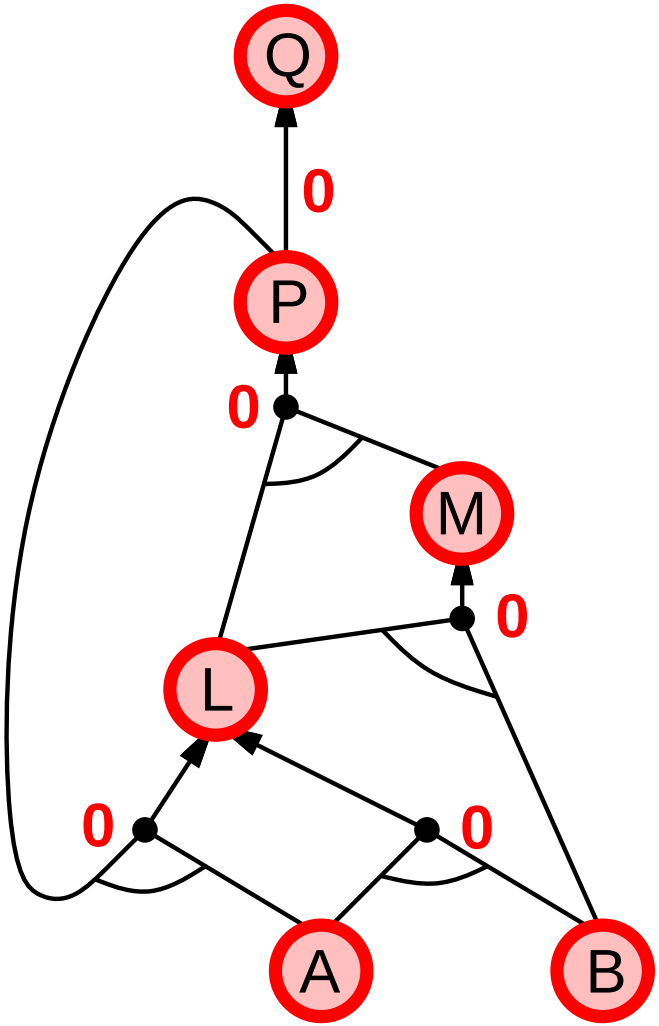
Forward chaining example



Forward chaining example



Forward chaining example



Backward chaining

Idea: work backwards from the query q:

to prove q by BC,

check if q is known already, or

prove by BC all premises of some rule concluding q

Avoid loops: check if new subgoal is already on the goal stack

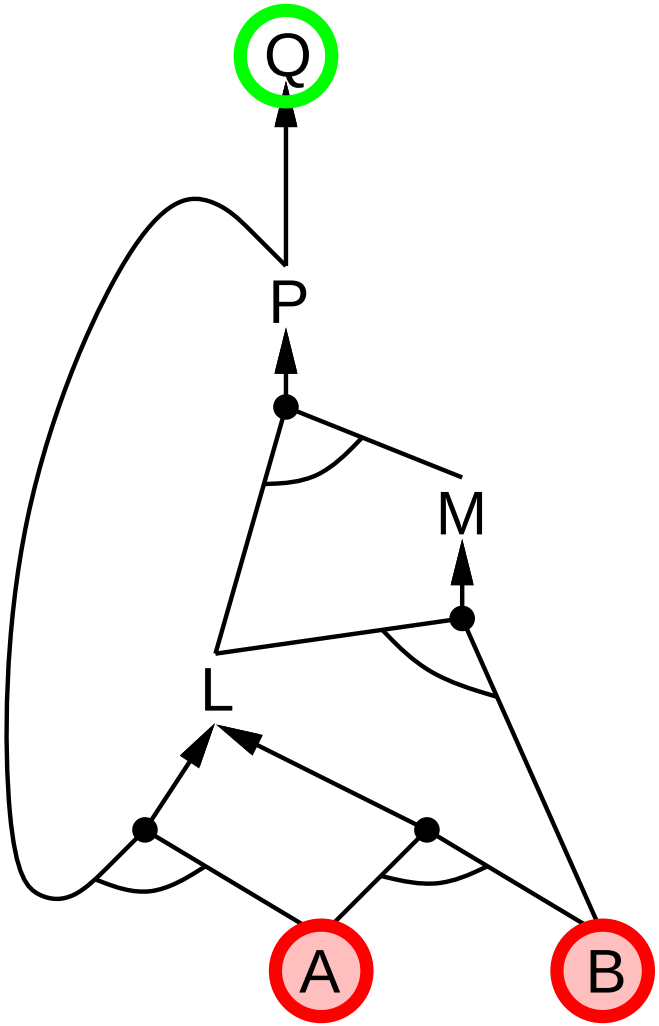
Avoid repeated work: check if new subgoal

1) has already been proved true, or

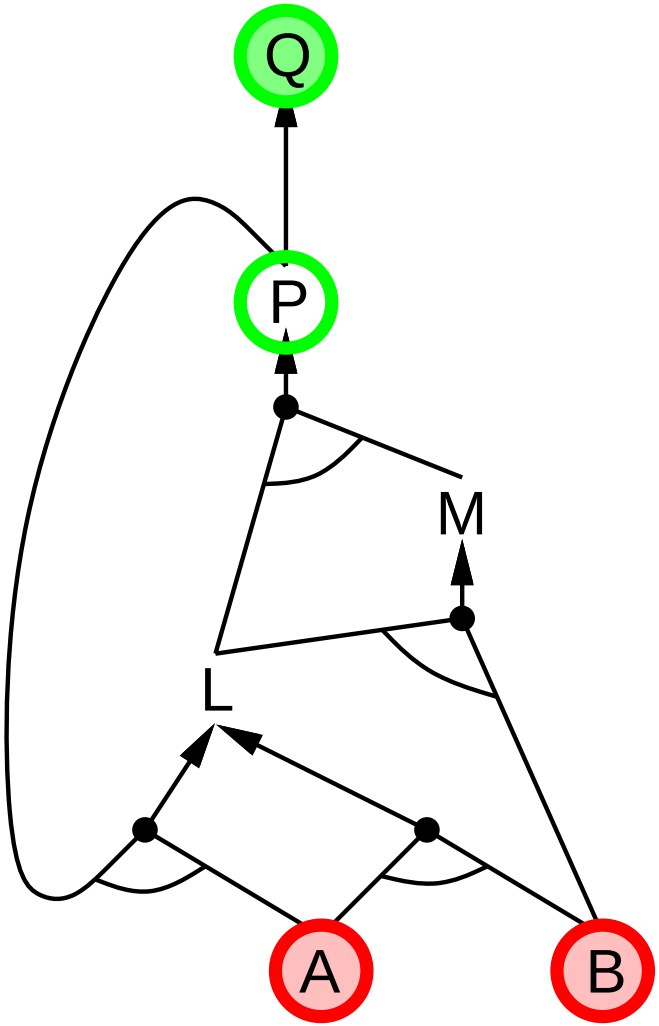
2) has already failed

The algorithm is essentially identical to the AND-OR-GRAPH search given in Fig. 4.11.

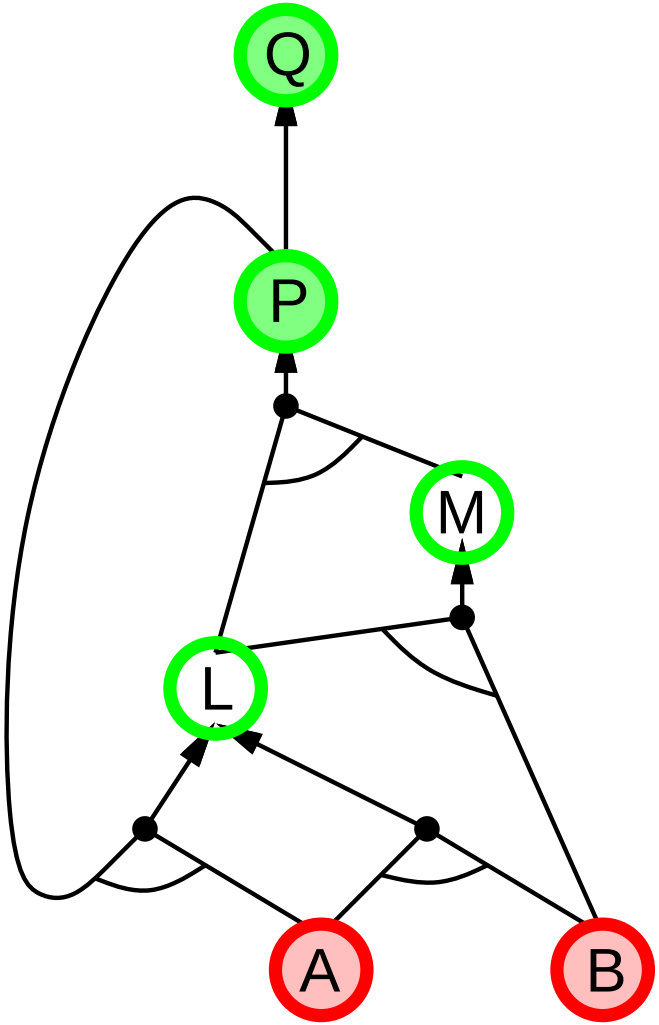
Backward chaining example



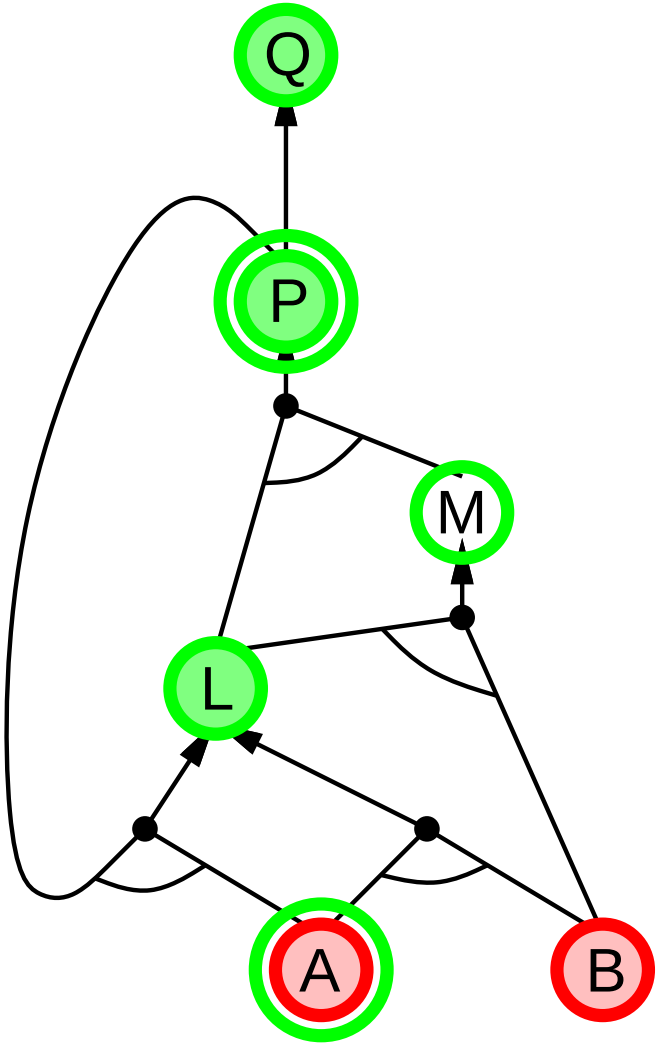
Backward chaining example



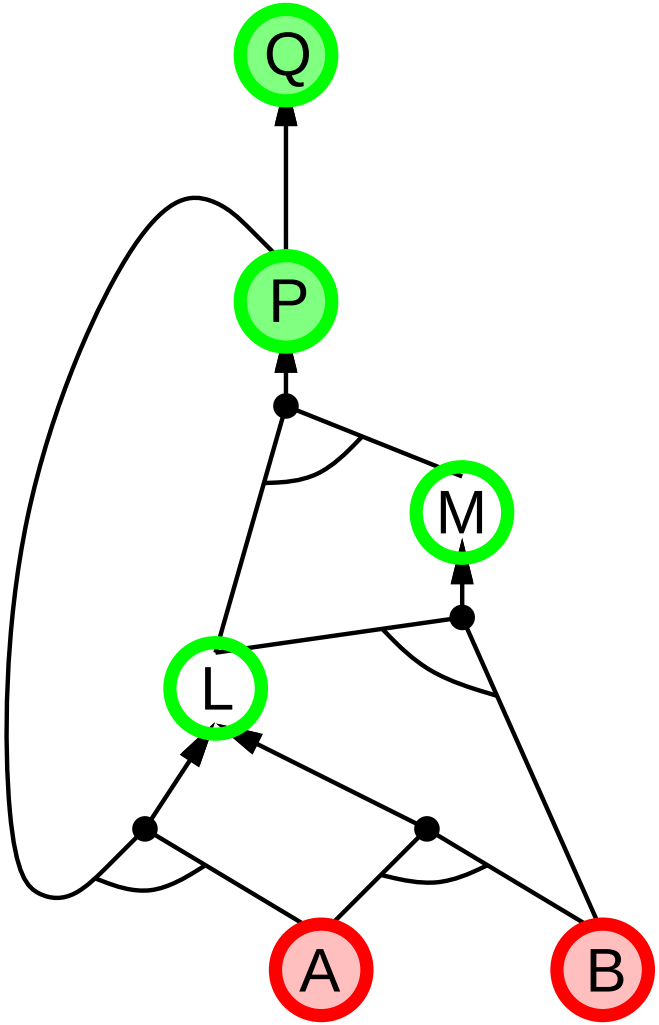
Backward chaining example



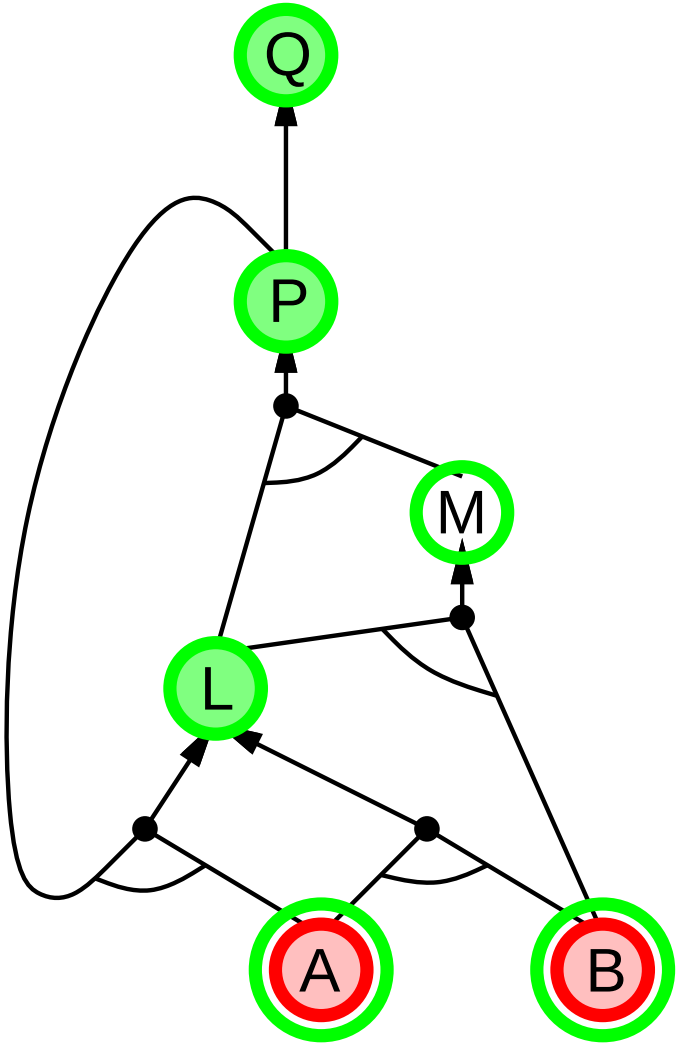
Backward chaining example



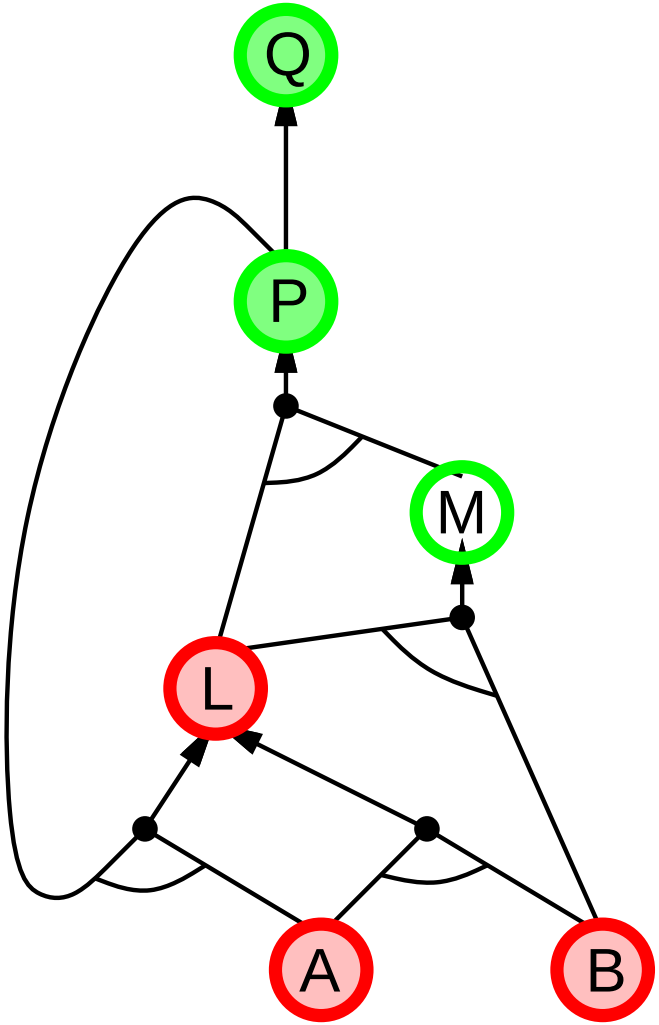
Backward chaining example



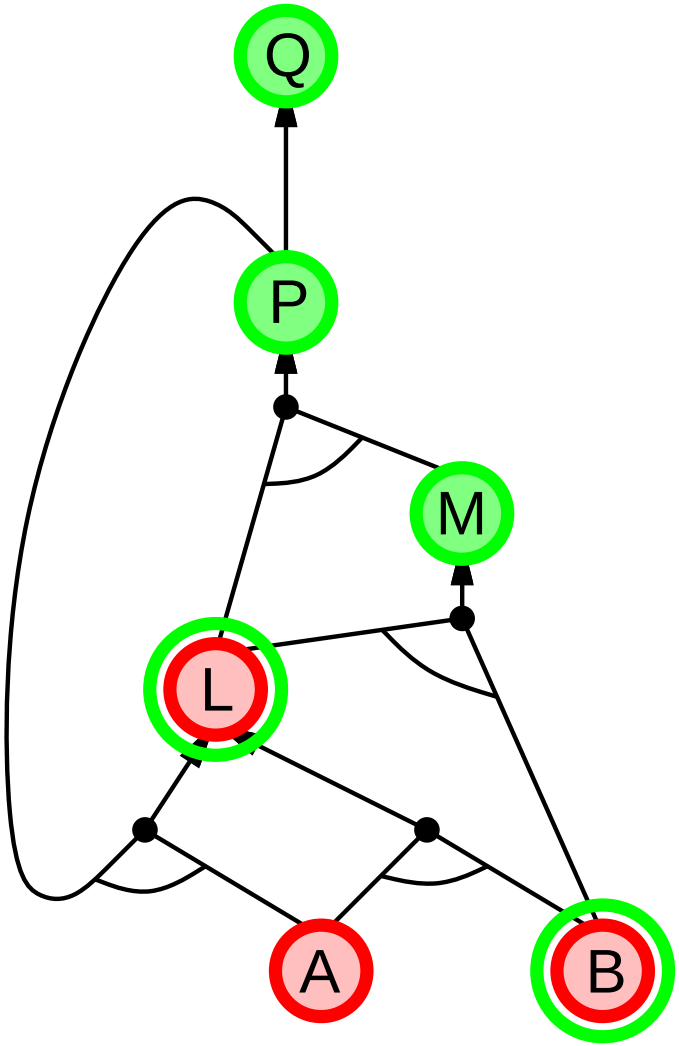
Backward chaining example



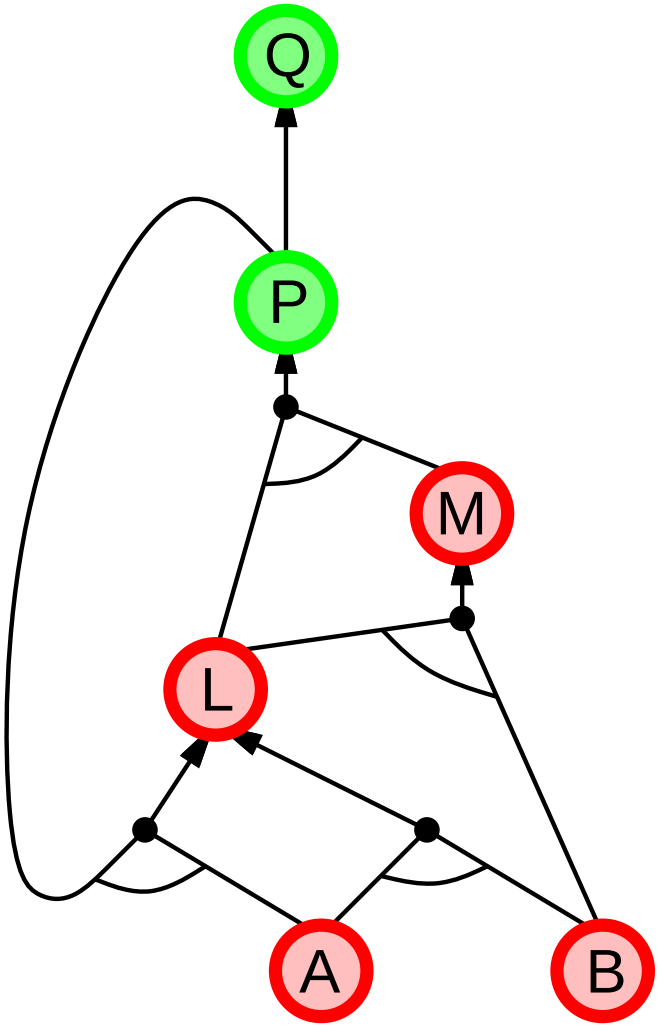
Backward chaining example



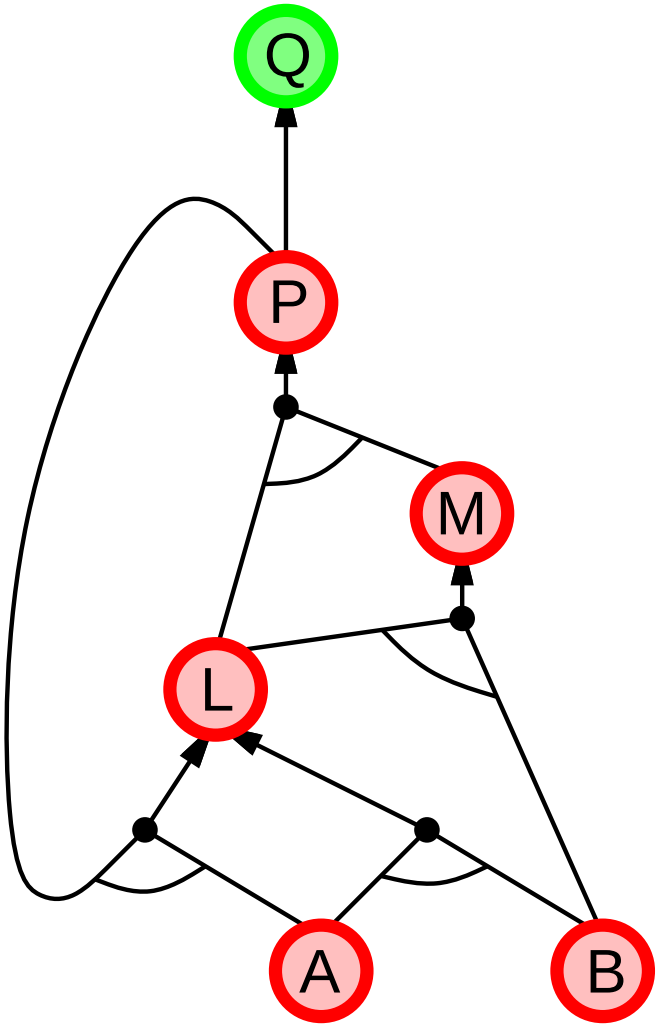
Backward chaining example



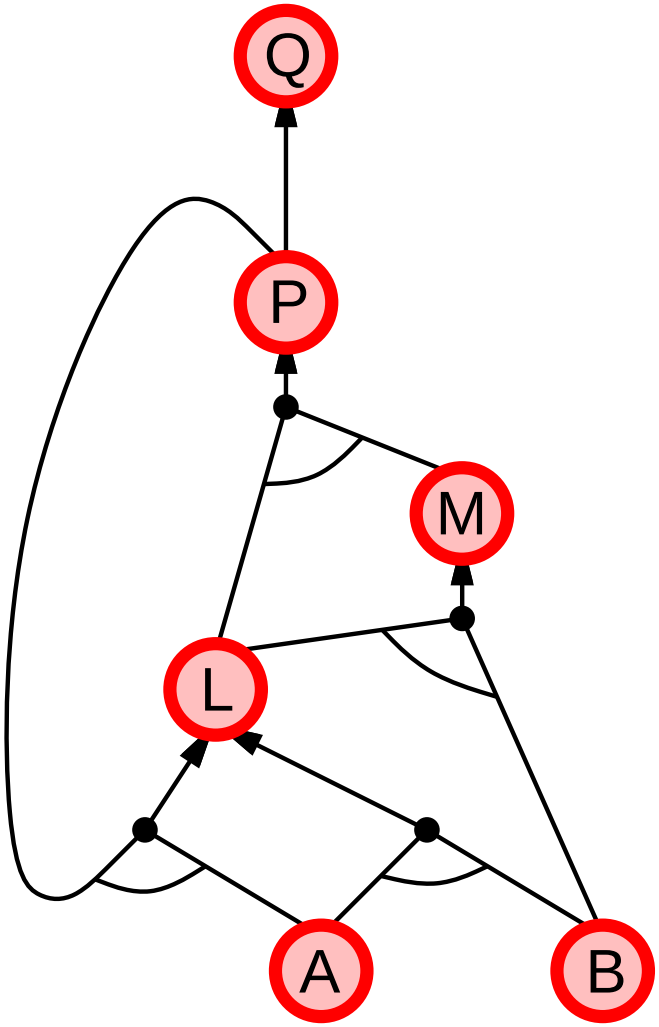
Backward chaining example



Backward chaining example



Backward chaining example



Forward vs. backward chaining

FC is data-driven, cf. automatic, unconscious processing,
e.g., object recognition, routine decisions

May do lots of work that is irrelevant to the goal

BC is goal-driven, appropriate for problem-solving,
e.g., Where are my keys? How do I get into a PhD program?

Complexity of BC can be *much less* than linear in size of KB

Satisfiability Problem

Satisfiability problem (is this sentence satisfiable?) is at the core of many general problems (e.g. constraint satisfaction problems).

Satisfiability can be checked by enumerating the possible models until one is found that satisfies the sentence using depth-first enumeration of possible models.

However, determining the satisfiability of sentences in propositional logic was the first problem proved to be NP-complete (in the worst case, it doesn't have a polynomial time algorithm).

Davis-Putnam (or DPLL) Algorithm

♢ Recursive, depth-first enumeration of possible models similar to TT-ENTAILS

♢ with important improvements such as early termination...

DPLL algorithm is one of the fastest satisfiability algorithms, despite the fact that it is very old. It can solve hardware verification problems with a million variables.

Local search algorithms

We can use local search algorithms for satisfiability problems, as well.

Since each clause in the sentence needs to be satisfied, evaluation function should count the number of unsatisfied clauses (e.g. min-conflict heuristic).

WALKSAT algorithm is one of the most effective algorithms.

WALKSAT algorithm

♢ In each iteration, pick an unsatisfied clause and picks a symbol to flip.

♢ Two ways to decide how to choose the symbol:

- randomly select the symbol from the clause and flip it with probability p

- pick one that minimizes the number of unsatisfied clauses in the new state

Local search algorithms

In underconstrained situations, solutions are easy to find.

Ex. A 3-CNF with 5 symbols and 5 clauses:

$(\neg D \vee \neg B \vee C) \wedge (B \vee \neg A \vee \neg C) \wedge (\neg C \vee \neg B \vee E)$
$\wedge (E \vee \neg D \vee B) \wedge (B \vee E \vee \neg C)$

How many solutions?

What makes it a hard satisfiability problem?

What if WALKSAT returns failure?

Local search algorithms

In underconstrained situations, solutions are easy to find.

Ex. A 3-CNF with 5 symbols and 5 clauses:

$(\neg D \vee \neg B \vee C) \wedge (B \vee \neg A \vee \neg C) \wedge (\neg C \vee \neg B \vee E)$
$\wedge (E \vee \neg D \vee B) \wedge (B \vee E \vee \neg C)$

How many solutions?

16 of the 32 possible worlds are models for this sentence!

What makes it a hard satisfiability problem?

How constrained it is (w.r.t number of constraints and number of variables)

What if WALKSAT returns failure?

We cannot tell if the sentence is unsatisfiable or just WALKSAT needs more time.

Shortcomings of PL

Lack of Expressive Efficiency:

We need to state the general rule about breeze and pits as a separate sentence for each square:

$B11 \Rightarrow P21 \vee P12$

$B21 \Rightarrow P11 \vee P22 \vee P31$

Shortcomings of PL - Fluents

Fluents: changing aspects of the world

E.g. Keeping track of location and orientation.

Let L_{ij} be a location proposition. Then we would like to say something like:

$L_{11} \wedge FacingRight \wedge Forward \Rightarrow L21$

But then we will have both L_{11} and $L21$ in the database!

Shortcomings of PL - Fluents

Solution: Associate propositions with time steps:

$L_{11}^0, FacingEast^0, HaveArrow^0 ...$

After associating time with prepositions, we need axioms to keep track of these fluents:

$L_{xy}^t \Rightarrow (Breeze^t \Leftrightarrow Breeze_{xy})$

Effect axioms:

$L_{11}^0 \wedge FacingEast^0 \wedge Forward^0 \Rightarrow (L_{21}^1 \wedge \neg L_{11}^1)$

We would need one such sentence for each possible time step and each possible square and orientation!

Shortcomings of PL - Frame Problem

We still have other problems. The effect axioms fail to state what is *unchanged* as the result of an action.

E.g. That the agent still has an arrow.

One possible solution is to add **frame axioms** explicitly asserting all the propositions that remain the same:

$$Forward^t \Rightarrow (HaveArrow^t \Leftrightarrow HaveArrow^{t+1})$$

$$Forward^t \Rightarrow (WumpusAlive^t \Leftrightarrow WumpusAlive^{t+1})$$

...

rtcomings of Propositional Logic - Representational Frame Problem

The proliferation of frame axioms is very inefficient and this problem was a significant problem for AI researchers.

Solution: Focusing from writing axioms about actions to writing axioms about fluents (**successor-state axioms**): $HaveArrow^{t+1} \Leftrightarrow (HaveArrow^t \wedge \neg Shoot^t)$

$$L_{11}^{t+1} \Leftrightarrow (L_{11}^t \wedge (\neg Forward^t \vee Bump^{t+1})) \\ \vee (L_{12}^t \wedge (\neg FacingSouth^t \wedge Forward^t)) \\ \vee (L_{21}^t \wedge (\neg FacingWest^t \wedge Forward^t)).$$

Summary

Logical agents apply inference to a knowledge base
to derive new information and make decisions

Basic concepts of logic:
- syntax: formal structure of sentences
- semantics: truth of sentences wrt models
- entailment: necessary truth of one sentence given another
- inference: deriving sentences from other sentences
- soundess: derivations produce only entailed sentences
- completeness: derivations can produce all entailed sentences

Wumpus world requires the ability to represent partial and negated information, reason by cases, etc. Propositional logic suffices for some of these tasks.

Truth table method is sound and complete for propositional logic.

Forward, backward chaining are linear-time, complete for Horn clauses

Resolution is complete for propositional logic.

Propositional logic lacks expressive power